

Academic Year 2025 – 2026

University: University of Pisa / Sant'Anna School of Advanced Studies

Course: Statistical Learning & Large Data

Supervisor(s): Prof.ssa Francesca Chiaromonte



SLLD - MODULE 2

---

# Regularized and Stable Lexical Classification in High-Dimensional Semantic Feature Spaces

---

**Michela Deodati & Leonardo Pratelli**

22.05.2026

## Abstract

This project reformulates the semantic rating dataset of Binder et al. (Binder et al., 2016) as a multi-class statistical learning problem. English lexical items are represented through 65 experiential semantic attributes and classified into three lexical-semantic classes: *Thing*, *Action*, and *Property*. After preprocessing, imputation, marginal distribution analysis, and standardization, the data are first explored through PCA and clustering to assess whether the semantic feature space contains interpretable structure and whether natural groups correspond to the known labels. The supervised analysis then uses multinomial generalized linear models as a baseline and investigates the effects of multicollinearity, feature screening, and regularization. In the high-dimensional setting, obtained by expanding the predictors with quadratic terms and pairwise interactions, marginal screening is used to reduce the feature space before fitting LASSO and Elastic Net models. Performance is evaluated through accuracy, balanced accuracy, macro-F1, class-wise recall, and confusion matrices, while sparsity patterns are inspected to identify the semantic predictors most relevant for lexical-class discrimination. The project ultimately assesses whether high-dimensional statistical learning can improve classification while preserving semantic interpretability.

**Keywords:** Statistical learning, high-dimensional data, semantic features, regularization, feature screening, lexical classification

---

*This report was prepared for the Statistical Learning for Large Data exam. The work has roots on the previous work "Unveiling Word Meaning: A statistical study on primitive semantic features", developed by Davide Testa, and reformulates it as a high-dimensional statistical learning study focused on large feature spaces, regularization, feature screening, class imbalance and stability.*

## Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                    | <b>4</b>  |
| <b>2</b> | <b>Dataset</b>                                                         | <b>6</b>  |
| <b>3</b> | <b>Methods</b>                                                         | <b>8</b>  |
| 3.1      | I. Data Preprocessing . . . . .                                        | 8         |
| 3.2      | II. Unsupervised exploration of the semantic space . . . . .           | 11        |
| 3.3      | III. Supervised classification on the original feature space . . . . . | 15        |
| 3.4      | IV. High-dimensional expansion and feature screening . . . . .         | 20        |
| 3.5      | V. Alternative classifiers . . . . .                                   | 21        |
| 3.6      | VI. Final model comparison and interpretation . . . . .                | 23        |
| <b>4</b> | <b>Results</b>                                                         | <b>24</b> |
| <b>5</b> | <b>Discussion</b>                                                      | <b>24</b> |
| <b>6</b> | <b>Conclusion</b>                                                      | <b>25</b> |
|          | <b>References</b>                                                      | <b>26</b> |
| <b>A</b> | <b>Original clustering and biclustering analyses</b>                   | <b>27</b> |
| <b>B</b> | <b>Detailed explanation of the Python preprocessing pipeline</b>       | <b>28</b> |
| B.1      | Phase 1.1: Initial dataset cleaning . . . . .                          | 29        |
| B.2      | Phase 1.2: Stratified train–test split . . . . .                       | 33        |
| B.3      | Phase 1.3: Imputation and standardization . . . . .                    | 37        |
| B.4      | Overall logic of the preprocessing pipeline . . . . .                  | 42        |
| <b>C</b> | <b>Detailed explanation of the Python analysis pipeline</b>            | <b>43</b> |
| C.1      | Phase 2: Unsupervised exploration of the semantic space . . . . .      | 44        |
| C.2      | Phase 3.1: Supervised data loading and preliminary checks . . . . .    | 46        |
| C.3      | Phase 3.2: Multinomial logistic baseline . . . . .                     | 48        |
| C.4      | Phase 3.3: Multicollinearity diagnostics . . . . .                     | 50        |
| C.5      | Phase 3.4: Feature screening . . . . .                                 | 52        |
| C.6      | Phase 3.5: Regularized multinomial models . . . . .                    | 53        |
| C.7      | Phase 3.6: Sparsity analysis . . . . .                                 | 54        |

|                                                                           |    |
|---------------------------------------------------------------------------|----|
| C.8 Phase 3.7: Final model evaluation . . . . .                           | 55 |
| C.9 Phase 3-extra: Alternative classifiers . . . . .                      | 56 |
| C.10 Phase 4: Supervised classification using clustering labels . . . . . | 57 |
| C.11 Overall logic of the post-preprocessing pipeline . . . . .           | 58 |
| C.12 Additional tables and figures . . . . .                              | 59 |

# 1 Introduction

The present work builds on the previous project *Unveiling Word Meaning: A statistical study on primitive semantic features*, developed by Davide Testa. Testa's work was concerned with the statistical analysis of semantic representations derived from the dataset introduced by Binder et al. (Binder et al., 2016). In the original project, Testa approached the dataset from the perspective of lexical semantics, psycholinguistics, and neurocognitive theories of meaning representation. His main objective was to assess whether a componential, brain-inspired representation of word meaning could capture meaningful semantic structures and support the distinction between broad lexical classes. Testa is publicly listed as a PhD Student in Artificial Intelligence at Sapienza Università di Roma in collaboration with Fondazione Bruno Kessler (FBK) (Testa, 2026). The dataset used in the original project consists of English lexical items represented through a set of primitive semantic attributes. After removing duplicates and reducing the data to the variables relevant to the semantic analysis, Testa worked with 534 lexical units and 65 semantic features, plus a qualitative target variable indicating the lexical-semantic type of each item. The target was organized into three main classes: *Things*, corresponding to nouns; *Actions*, corresponding to verbs; and *Properties*, corresponding to adjectives. The resulting class distribution was highly unbalanced, with 434 items classified as Things, 56 as Actions, and 44 as Properties. This imbalance already represented an important statistical issue, since classification accuracy alone could be strongly influenced by the majority class. The original analysis was structured around two main research questions. The first asked whether similarity structures could be identified from the semantic information encoded in the 65 primitive features. The second asked whether morpho-syntactic classes could be distinguished solely on the basis of semantic information. To address these questions, Testa developed a pipeline including data preprocessing, unsupervised learning, dimensionality reduction, supervised classification, and feature selection. In the preprocessing stage, missing values were identified and imputed using the mean of the corresponding feature. Since several variables showed a right-skewed distribution and the presence of outliers, a logarithmic transformation was applied to reduce skewness and make the data more suitable for subsequent statistical analysis. The features were then normalized so that variables measured on different scales or ranges would contribute more evenly to distance-based and model-based procedures. A preliminary correlation analysis showed substantial multicollinearity among several semantic dimensions. In particular, Testa identified groups of highly correlated features related to visual and somatosensory attributes, auditory attributes, temporal and causal attributes, and social-emotional attributes. Rather than eliminating or merging correlated features, the original project kept them as separate predictors, because one of its goals was precisely to evaluate the contribution of individual primitive semantic dimensions to word meaning. The unsupervised part of the project investigated whether the semantic feature space contained interpretable similarity structures. Testa applied k-means clustering, agglomerative hierarchical clustering, and biclustering. The k-means analysis produced only partially satisfactory results: with two clusters, the distinction was too coarse and difficult to interpret, while with ten clusters some semantic groupings emerged but remained noisy. Hierarchical clustering led to qualitatively more interpretable groupings, including categories such as food, animals, musical instruments, and sentiment-related words. Biclustering was introduced as a more flexible method, since it simultaneously groups rows and columns and can therefore identify local semantic patterns involving subsets of words and subsets of features. This analysis produced a large number of semantic biclusters, including groups interpretable as Time, Food and Drinks, Animals, Sounds, Human Beings, Practical Tools, Locations, and Negative Events and Feelings. Principal Component Analysis was then used as a dimensionality reduction and denoising strategy. In Testa's interpretation, PCA helped reveal latent semantic dimensions underlying the original feature space. The first ten principal components accounted for approximately 75% of the total variance, while the first two components accounted for about 36%. These first two dimensions were interpreted as a broad opposition between *Concreteness* and *Abstractness*. When clustering was performed on the PCA-reduced representation, the results became clearer than those obtained in the original feature space, suggesting that PCA could reduce noise and highlight more stable semantic structures. The supervised part of the original project addressed the classification of lexical-semantic classes. Testa used multinomial logistic

regression with a LASSO penalty, implemented through the `glmnet` framework, to predict whether each lexical item belonged to the class *Thing*, *Action*, or *Property*. Initial models were trained on the 65 original semantic features and on PCA-reduced representations. The LASSO model using all 65 features achieved an accuracy around 0.84, while models based on two or ten principal components achieved slightly lower accuracies. Since the classes were strongly unbalanced, several strategies were explored, including oversampling, cross-validation with the selection of an optimal regularization parameter, stratified cross-validation, and alternative train-test partitions. Some configurations produced very high accuracies, but Testa noted that these results could be affected by overfitting or by the instability induced by the class imbalance. In the final feature selection step, the LASSO coefficients were used to identify a reduced set of relevant predictors. A model fitted on 32 selected features out of the original 65 achieved an accuracy around 0.91. Overall, Testa concluded that the componential semantic representation proposed by Binder et al. captures a considerable amount of semantic and relational information. His results suggested that the 65 primitive semantic features are informative both for identifying broad semantic similarity structures and for distinguishing among lexical classes. The PCA analysis was interpreted as especially relevant, because it appeared to reveal a latent concrete/abstract opposition that may play an important role in the organization of word meaning. At the same time, the original project remained mainly exploratory: it focused on validating the usefulness of the semantic features, rather than systematically studying the statistical consequences of high dimensionality, multicollinearity, class imbalance, and model instability. The present project resumes the analysis from this point and reformulates it within the framework of statistical learning for high-dimensional semantic data. Rather than treating the Binder representation only as an object of exploratory semantic analysis, we organize the work around a structured pipeline that combines preprocessing, unsupervised exploration, supervised classification, multicollinearity diagnostics, feature screening, regularization, and final model comparison. The goal is therefore not simply to verify whether the 65 primitive semantic attributes are informative, but to assess how different statistical learning strategies behave when lexical-semantic classification is affected by redundancy among predictors, class imbalance, and the possible expansion of the feature space. The first part of the project focuses on data preparation and exploratory analysis. Missing values are inspected and handled through a coherent imputation strategy, while the marginal distributions of the semantic variables are examined in order to detect skewness, low-variance predictors, and possible outlying patterns. The resulting feature matrix is then standardized when required, especially before applying PCA, clustering, and regularized models. The unsupervised analysis uses Principal Component Analysis to study the main directions of variation in the semantic space and clustering methods to evaluate whether the observations form natural groups. These clusters are compared with the three known lexical-semantic labels in order to understand whether the geometry of the semantic feature space reflects, at least partially, the distinction between *Thing*, *Action*, and *Property*. The supervised part of the project frames the task as a multi-class classification problem and uses multinomial generalized linear models as the main statistical reference. A simple baseline model is first constructed to establish a point of comparison. The analysis then explicitly addresses multicollinearity among the semantic predictors through correlation analysis, variance inflation diagnostics, partial association measures, and dendrogram-based inspection of redundant feature groups. When the predictor space is expanded through quadratic terms and pairwise interactions, the problem becomes high-dimensional, with  $p > n$ . For this reason, marginal feature screening is introduced as an intermediate step to reduce the number of predictors before fitting regularized multinomial models. Finally, the project compares LASSO and Elastic Net as regularized classification strategies. LASSO is used to obtain sparse models and identify the most informative semantic predictors, while Elastic Net is used to combine sparsity with greater robustness in the presence of correlated variables. Model performance is evaluated using accuracy, balanced accuracy, macro-F1, class-wise recall, and confusion matrices, so that the strong imbalance among the lexical-semantic classes does not make raw accuracy the only criterion of evaluation. Particular attention is given to the sparsity patterns produced by the models: the analysis examines how many coefficients are set to zero, which semantic features or expanded terms are retained, and whether the selected predictors offer an interpretable account of the distinction between lexical classes. In this way, the project extends Testa's original semantic analysis into a statistical learning

study focused on preprocessing, unsupervised structure, multinomial classification, multicollinearity, screening, regularization, sparsity, and interpretability in semantic data.

## 2 Dataset

The dataset used in this project is derived from the semantic ratings introduced by Binder et al. (Binder et al., 2016) in the paper *Toward a Brain-Based Componential Semantic Representation*. The aim of that work was to provide a componential representation of word meaning in which lexical concepts are described through a set of experiential semantic dimensions. Unlike distributional semantic models, where words are represented on the basis of their textual co-occurrence patterns, this dataset represents each lexical item through human ratings on theoretically motivated semantic attributes. These attributes are intended to capture the extent to which a word meaning involves perceptual, motor, spatial, temporal, social, affective, attentional and cognitive experience.

The dataset contains approximately 535 English lexical entries, each described by 65 semantic attributes. Each row corresponds to a word or word-sense, while each of the 65 core columns corresponds to a semantic dimension. The values are continuous ratings, interpreted as the average degree to which a specific attribute contributes to the meaning of the word. A high value indicates that the corresponding experiential dimension is strongly involved in the concept, whereas a low value indicates that the dimension is weakly involved or not relevant. For example, a word such as *yellow* is expected to receive high ratings on visual dimensions such as *Vision*, *Color* and *Bright*, while a word such as *agreement* is expected to be more strongly associated with dimensions such as *Social*, *Communication* and *Cognition*. Similarly, an action-related word may show high values on dimensions such as *Motion*, *Body*, *UpperLimb*, *LowerLimb* or *Practice*. The expression “Binder et al. 2016” therefore refers specifically to the semantic rating dataset associated with the paper by Binder and colleagues. Although the representation is described as brain-based because the semantic dimensions are motivated by neurocognitive theories of conceptual representation, the numerical values used in the dataset are not direct neuroimaging measurements. They are behavioral semantic ratings collected from human participants and organized into a structured word-by-feature matrix. In this sense, the dataset provides a bridge between cognitive theories of meaning and statistical learning, since it translates qualitative semantic properties into quantitative predictors. The 65 semantic attributes cover a broad range of experiential domains. They include visual features such as *Vision*, *Color*, *Shape*, *Bright*, *Dark*, *Pattern*, *Large* and *Small*; auditory features such as *Audition*, *Sound*, *Music*, *Speech*, *Loud*, *Low* and *High*; somatic and motor features such as *Touch*, *Temperature*, *Texture*, *Weight*, *Pain*, *Head*, *UpperLimb*, *LowerLimb* and *Practice*; spatial and temporal features such as *Landmark*, *Path*, *Scene*, *Near*, *Toward*, *Away*, *Time*, *Duration*, *Long* and *Short*; and more abstract dimensions such as *Caused*, *Consequential*, *Social*, *Human*, *Communication*, *Self*, *Cognition*, *Benefit*, *Harm*, *Pleasant*, *Unpleasant*, *Happy*, *Sad*, *Angry*, *Disgusted*, *Fearful*, *Surprised*, *Drive*, *Needs*, *Attention* and *Arousal*. This organization makes the dataset particularly suitable for studying whether broad lexical classes can be predicted from structured semantic information.

In addition to the semantic attributes, the original data file contains auxiliary lexical and categorical variables, such as word identifiers, word class codes, frequency-related variables and category labels. However, in this project we use only the 65 semantic attributes as predictors. Lexical variables such as word length, frequency or imageability are excluded from the main predictive models, because the objective is to evaluate the statistical information contained in the semantic representation itself. The response variable is defined as a lexical-semantic class, with three target categories: *Thing*, *Action* and *Property*. This follows the structure already adopted in the previous *Unveiling Word Meaning: A statistical study on primitive semantic features* project, where nouns were associated with things, verbs with actions, and adjectives or property-like items with properties.

A relevant characteristic of the dataset is the strong imbalance among classes. The majority of the observations belong to the class *Thing*, while *Action* and *Property* are much less represented. This

imbalance has important consequences for model evaluation. A classifier may obtain a high raw accuracy by systematically favoring the majority class, while performing poorly on the minority classes. For this reason, accuracy alone is not sufficient to assess the quality of the models. In the following analyses, performance will therefore be evaluated using metrics that are more appropriate for imbalanced classification, including balanced accuracy, macro-F1, class-wise recall and confusion matrices.

The dataset also contains a limited number of missing values. These missing values are not necessarily random errors, since some semantic attributes were not equally meaningful or available for all grammatical classes. In the present project, missing values are handled through imputation. To avoid data leakage, the imputation parameters are estimated only on the training set and then applied to the test set. The same principle is followed for standardization: means and standard deviations are computed from the training data only, and the resulting transformation is then applied to both training and test observations.

The original semantic representation has 65 predictors and can be considered a moderate-dimensional dataset. However, one of the objectives of this project is to reformulate it as a high-dimensional statistical learning problem. To this end, we construct an expanded feature space starting from the original semantic matrix. Let

$$X = (X_1, X_2, \dots, X_{65})$$

denote the original vector of semantic attributes for a given word. The expanded representation includes the original features, the quadratic terms

$$X_1^2, X_2^2, \dots, X_{65}^2,$$

and all pairwise interaction terms

$$X_j X_k, \quad j < k.$$

The resulting number of predictors is

$$65 + 65 + \binom{65}{2} = 65 + 65 + 2080 = 2210.$$

This expansion allows the models to capture not only the individual contribution of each semantic dimension, but also possible non-linear effects and interactions between semantic attributes. For instance, an interaction such as *Vision*  $\times$  *Color* may capture a specifically visual-perceptual profile, while *Motion*  $\times$  *Body* may be relevant for action-related words, and *Social*  $\times$  *Emotion* may characterize more abstract or interpersonal meanings.

After feature expansion, the problem becomes high-dimensional, since the number of predictors exceeds the number of observations:

$$p = 2210 > n.$$

This setting is statistically challenging. The original 65 semantic features already contain natural correlations, because related experiential dimensions tend to co-occur in word meanings. For example, visual dimensions may be correlated with each other, social and emotional dimensions may overlap, and motor dimensions may jointly characterize action concepts. The addition of squared terms and pairwise interactions further increases redundancy and multicollinearity. As a result, ordinary unregularized multinomial regression is expected to be unstable or infeasible, and the risk of overfitting becomes substantial.

For this reason, the dataset is well suited to the methodological aims of the project. The original 65-feature representation allows us to build baseline models and evaluate whether semantic attributes alone are informative for lexical classification. The expanded 2210-feature representation allows us to study a genuine large-feature-space problem, where regularization, dimension reduction, feature screening, rebalancing and stability analysis become necessary. In particular, the project will compare models fitted on the original semantic matrix with high-dimensional penalized models, PCA-based dimensionality reduction, and a two-stage strategy based on marginal feature screening followed by penalized multinomial regression.

To summarize, the Binder et al. dataset provides a structured and cognitively motivated representation of word meaning. In its original form, it is a semantic rating dataset composed of English lexical items and 65 experiential attributes. In the present work, it is used as the basis for a high-dimensional statistical learning problem. By expanding the semantic feature space and studying the resulting classification task under multicollinearity and class imbalance, the project aims to identify which semantic dimensions and interactions contribute most reliably to the prediction of lexical-semantic classes.

### 3 Methods

All analyses are implemented in Python, using a modular pipeline organized into successive phases. Each phase produces intermediate datasets, diagnostic files, and summary reports, so that the whole workflow remains reproducible and inspectable. The methodological structure follows a progressive logic: we first prepare the data, then explore the semantic space without using the labels, and finally formulate the supervised classification problem, moving from simple baseline models to regularized and high-dimensional approaches.

#### 3.1 I. Data Preprocessing

The preprocessing phase is designed to produce a clean, reproducible, and statistically usable version of the Binder semantic rating dataset before any exploratory or predictive analysis is performed. All operations are implemented in Python and organized into three consecutive steps, corresponding to the outputs stored in folder I: initial dataset cleaning, stratified train–test splitting, and train-based imputation and standardization. The guiding principle of this phase is to preserve as much valid lexical information as possible while avoiding data leakage between training and test data. We start from the 65-feature version of the dataset, in which each observation corresponds to a lexical entry and each predictor corresponds to an experiential semantic attribute. The dataset contains three structural columns, namely `entry_id`, `word`, and `target_word_class`, plus 65 semantic features. The target variable is represented by three lexical-semantic classes: *noun*, *verb*, and *adjective*, corresponding respectively to the conceptual categories *Thing*, *Action*, and *Property*. In the first preprocessing script, we verify that all required columns are present, that the expected number of semantic predictors is equal to 65, and that the target variable contains only valid class labels. During the initial cleaning step, we do not apply any statistical transformation to the predictors. We only perform structural checks. Exact duplicated rows are searched for and removed if present, because exact duplicates would artificially increase the weight of repeated observations without adding information. In the present dataset, no exact duplicate rows are found. We also inspect duplicated word forms. This check identifies two rows corresponding to the word *used*, which appears once as an adjective and once as a verb. These rows are not removed, because the statistical unit of the analysis is the lexical entry rather than the orthographic word form alone. A repeated word form with a different lexical-semantic class is therefore considered a valid observation. The initial class distribution, reported in Table 1, confirms that the dataset is strongly imbalanced. Out of 535 lexical entries, 434 are nouns, 62 are verbs, and 39 are adjectives. This means that the majority class alone accounts for approximately 81.1% of the dataset. This imbalance is important methodologically, because a naive classifier predicting only the majority class would already obtain high raw accuracy. For this reason, in the later supervised analyses, accuracy must be complemented by balanced accuracy, macro-F1, class-wise recall, and confusion matrices.



| Class     | Count | Percentage |
|-----------|-------|------------|
| noun      | 434   | 81.12%     |
| verb      | 62    | 11.59%     |
| adjective | 39    | 7.29%      |

Table 1 – Class distribution in the cleaned dataset before train–test splitting.

We then examine missing values feature by feature. As shown in Table 2, missingness is limited to four semantic predictors: `complexity`, `practice`, `caused`, and `drive`. The feature `complexity` contains the largest number of missing values, with 101 missing entries, corresponding to approximately 18.9% of the dataset. The features `practice` and `caused` each contain 39 missing entries, while `drive` contains only one missing value. Overall, 102 rows contain at least one missing value, while 433 rows are complete.

| Feature                 | Missing values | Missing percentage |
|-------------------------|----------------|--------------------|
| <code>complexity</code> | 101            | 18.88%             |
| <code>practice</code>   | 39             | 7.29%              |
| <code>caused</code>     | 39             | 7.29%              |
| <code>drive</code>      | 1              | 0.19%              |

Table 2 – Semantic features with missing values in the cleaned dataset.

We decide not to remove rows with missing values at this stage. This choice is motivated by two reasons. First, the dataset is relatively small, and deleting all incomplete rows would reduce the number of available observations from 535 to 433. Second, because the target distribution is already highly imbalanced, removing observations could further weaken the representation of the minority classes. We also decide not to remove the four features with missing values, because the missingness is concentrated and manageable, and because the 65 attributes define the original semantic representation that we want to preserve. Missing values are therefore retained after the initial cleaning step and handled later through imputation. After structural cleaning, we create a train–test split. The split is performed before imputation and standardization, because all preprocessing parameters must be estimated only on the training data. We use an 80/20 split with stratification on `target_word_class`. Stratification is necessary because the target classes are strongly unbalanced, and a purely random split could produce unstable or poorly representative minority-class partitions. The resulting training set contains 428 observations, while the test set contains 107 observations. As reported in Table 3, the class proportions are preserved very closely across the full dataset, training set, and test set.

| Split        | Class     | Count | Percentage |
|--------------|-----------|-------|------------|
| Full dataset | noun      | 434   | 81.12%     |
| Full dataset | verb      | 62    | 11.59%     |
| Full dataset | adjective | 39    | 7.29%      |
| Train        | noun      | 347   | 81.07%     |
| Train        | verb      | 50    | 11.68%     |
| Train        | adjective | 31    | 7.24%      |
| Test         | noun      | 87    | 81.31%     |
| Test         | verb      | 12    | 11.21%     |
| Test         | adjective | 8     | 7.48%      |

Table 3 – Class distribution after stratified train–test splitting.

The stratified split also allows us to compute a first reference baseline. Since the majority class is *noun*, a classifier that always predicts *noun* would obtain an accuracy of approximately 0.811 on the full dataset. This value is not used as a final modelling result, but it is important as a warning: any supervised model must be evaluated against this trivial baseline and must be judged with metrics that account for minority-class performance. The third preprocessing step consists of missing-value imputation and feature standardization. Both operations are fitted exclusively on the training set and then applied to the test set. This is a central methodological constraint: if imputation values or scaling parameters were computed using the whole dataset, information from the test set would enter the training pipeline, producing data leakage and an overly optimistic evaluation. For imputation, we use median imputation. The median is preferred because it is simple, reproducible, and more robust than the mean in the presence of asymmetry or outlying values. Since the variables are numeric semantic ratings, median imputation provides a conservative replacement strategy that preserves the scale of each feature without introducing model-based assumptions. The imputer is fitted on the 428 training observations. The median values learned from the training set are then used to impute both training and test data. As summarized in Table 4, before imputation, 81 training rows and 21 test rows contain at least one missing value. After imputation, both sets contain zero missing values.

| Split | Rows with missing values before imputation | Rows with missing values after imputation |
|-------|--------------------------------------------|-------------------------------------------|
| Train | 81                                         | 0                                         |
| Test  | 21                                         | 0                                         |

Table 4 – Effect of median imputation on missing values.

After imputation, we standardize all 65 semantic features using a standard score transformation. The scaler is fitted only on the imputed training set, estimating one mean and one standard deviation for each feature. These parameters are then applied to both training and test data. Standardization is required because several subsequent methods are scale-sensitive: PCA depends on variances and covariances, clustering and k-nearest-neighbor methods depend on distances, and regularized models such as Ridge, LASSO, and Elastic Net penalize coefficients in a way that is not invariant to predictor scale. Without standardization, features with larger numerical dispersion could dominate the analysis for purely metric reasons.

The standardization diagnostics confirm that the transformation behaves as expected on the training data. After scaling, the maximum absolute feature mean in the training set is approximately  $2.45 \times 10^{-16}$ , while the feature standard deviations are numerically equal to one, up to floating-point precision. This confirms that the standardized training matrix is centered and scaled correctly. The test set is not re-

centered independently; it is transformed using the training-set parameters, preserving its role as external evaluation data.

At the end of preprocessing, we obtain four main datasets: the raw train and test sets, the imputed train and test sets, and the standardized train and test sets. The imputed but unscaled datasets are useful for methods that do not require standardized predictors, whereas the standardized datasets are used for PCA, clustering, distance-based methods, and regularized classification. The preprocessing phase therefore produces a controlled and leakage-free foundation for all subsequent analyses.

### 3.2 II. Unsupervised exploration of the semantic space

Before moving to supervised classification, we carry out an unsupervised exploration of the standardised training set. The goal of this phase is diagnostic rather than predictive: following (James et al., 2013), unsupervised learning is used here as a form of exploratory data analysis, aimed at assessing whether the observations organise themselves into meaningful groups independently of the known class labels (James et al., 2013). Two complementary tools are applied: Principal Component Analysis (PCA) for dimensionality reduction and visualisation, and clustering to search for natural groupings.

#### Principal Component Analysis

PCA was applied to the standardised feature matrix  $\mathbf{X} \in \mathbb{R}^{428 \times 65}$ , where each row represents a lexical item and each of the 65 columns one semantic feature. Standardisation to zero mean and unit variance, performed in Phase 1, is a prerequisite for PCA, since otherwise features measured on different scales would dominate the principal directions (James et al., 2013).

**Variance explained.** We examine the proportion of variance explained (PVE) by each principal component and the cumulative PVE (James et al., 2013). Figure 1 shows the scree plot; Figure 2 shows the cumulative curve. Retaining the first **10 principal components** accounts for approximately **75%** of the total variance. The first two components alone explain roughly **36%** of the total variance ( $\text{cPVE}_2 \approx 0.36$ ), consistent with the high-dimensional and noisy nature of the semantic rating space. In accordance with the recommendation in (James et al., 2013), the number of components to retain was chosen by inspecting the elbow of the scree plot and selecting the smallest number that captures a substantial fraction of the total variation.

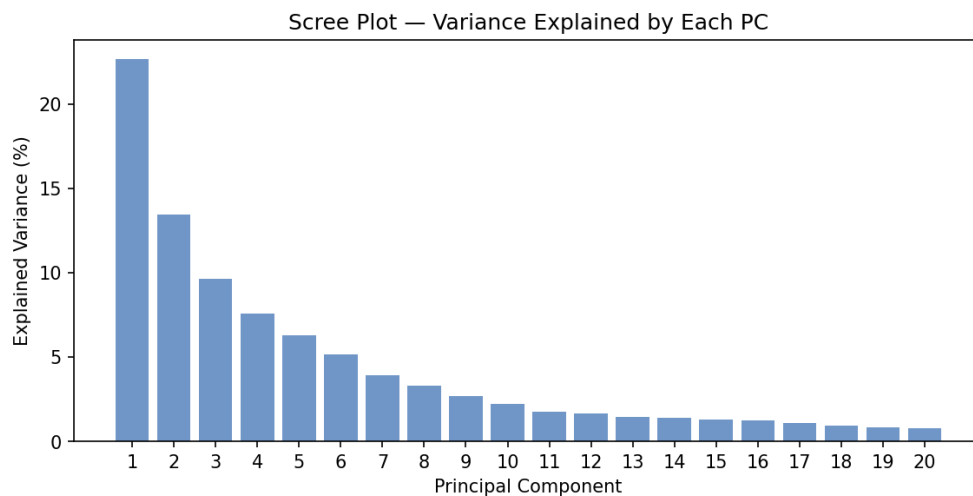


Figure 1 – Scree plot: proportion of total variance explained by each principal component. The contribution decreases steeply after the first few components.

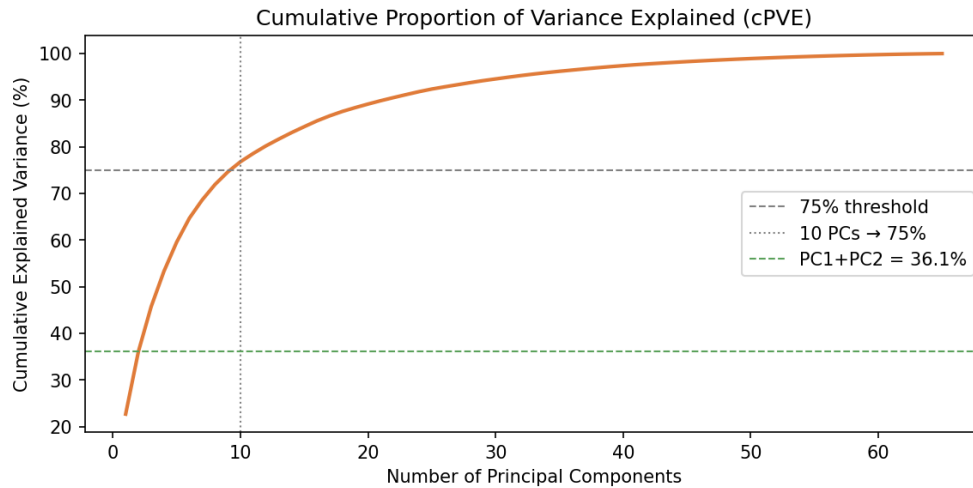


Figure 2 – Cumulative proportion of variance explained (cPVE). The dashed horizontal line marks the 75% threshold, reached at approximately 10 components.

**Projection onto the first two components.** Figure 3 shows the 428 training observations projected onto the plane defined by PC1 and PC2, coloured by morphosyntactic class. Following the biplot framework described in (James et al., 2013), the loading vectors reveal the semantic interpretation of each axis. PC1 is dominated by sensorimotor features (VISION, COLOUR, SHAPE, TOUCH, TEXTURE), tracing a *concreteness* gradient along which nouns (*Thing*) cluster toward the concrete pole. PC2 captures auditory and affective features (FAST, SOUND, PAIN, ATTENTION, LOUD), separating more dynamic or eventive concepts from static ones. A broad but imperfect separation between word classes is visible, suggesting that the feature space encodes class-relevant variation, though the boundaries are not sharp.

## Clustering

Clustering was performed on the PCA-reduced space retaining the first 10 principal components (those accounting for approximately 75% of the variance). Operating in this reduced space attenuates the effect of noise and high dimensionality on distance computations (James et al., 2013), which notes that many statistical techniques, including clustering, can benefit from using the first few principal component score vectors rather than the full feature matrix.

Two methods were applied: *K*-means clustering and Agglomerative Hierarchical Clustering (AHC) with complete linkage and Euclidean distance, both introduced in (James et al., 2013).

**Selecting the number of clusters.** The optimal *K* was investigated using two criteria.

1. **Elbow method** (*K*-means): the within-cluster inertia was plotted as a function of *K* (Figure 4). As noted in (James et al., 2013), minimising the within-cluster variation is the objective of *K*-means; the elbow in the inertia curve indicates where additional clusters yield diminishing returns. The Hartigan index, defined as  $H(K) = (\text{inertia}(K)/\text{inertia}(K+1) - 1)(n - K - 1)$ , formalises this criterion; both point to  $K = 2$  as the most parsimonious solution.
2. **Average silhouette score**: the mean silhouette coefficient was computed for  $K \in \{2, \dots, 15\}$  for both methods (Figures 5 and 6). The silhouette scores are modest throughout, reflecting the absence of strongly separated clusters in this data. The highest values are obtained at small *K*, with  $K = 2$  yielding approximately 0.43 for *K*-means and 0.16 for AHC.

Given the flat silhouette profile, two values of *K* are considered in the visualisations:  $K = 2$  (indicated by both criteria) and  $K = 10$  (a semantically richer partition). An important caveat noted in (James et al.,

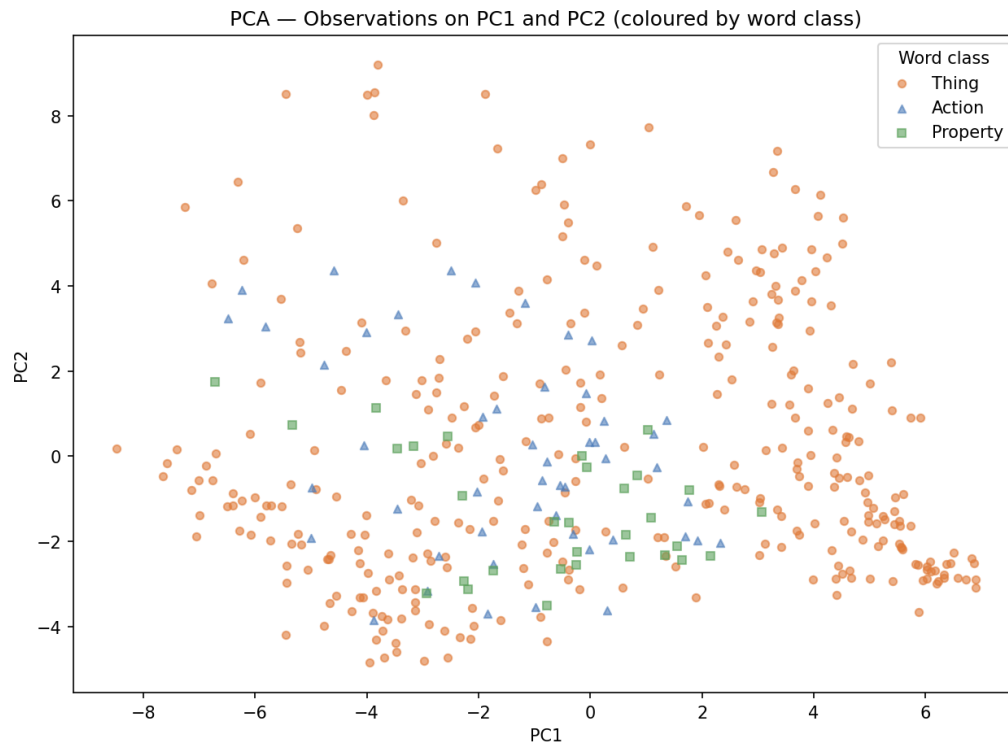


Figure 3 – Training observations projected onto PC1 and PC2. Colours indicate morphosyntactic class: *Thing* (nouns), *Action* (verbs), *Property* (adjectives). A concreteness gradient is visible along PC1.

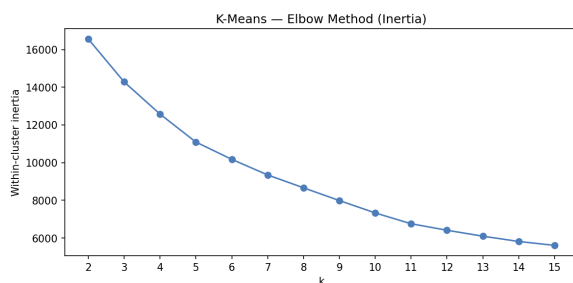


Figure 4 – K-means: within-cluster inertia (elbow method).

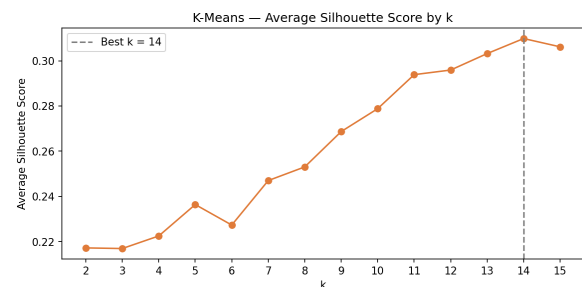


Figure 5 – K-means: average silhouette score by K.

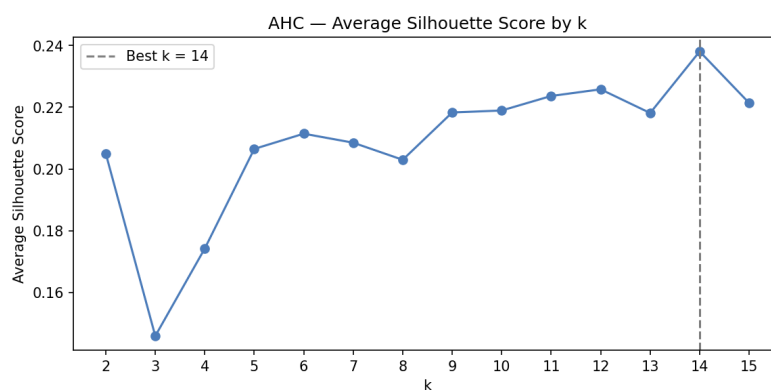


Figure 6 – AHC (complete linkage, Euclidean distance): average silhouette score by K.

2013) is that  $K$ -means finds a *local* rather than global optimum; for this reason the algorithm was run with 20 random initialisations and the solution minimising the objective was retained.

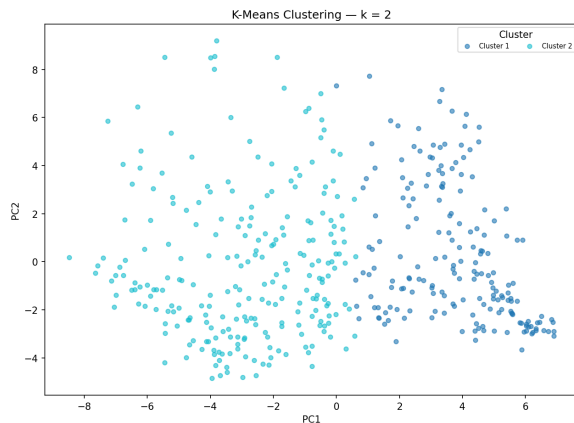


Figure 7 –  $K$ -means,  $K = 2$ : the partition roughly follows the concreteness gradient of PC1.

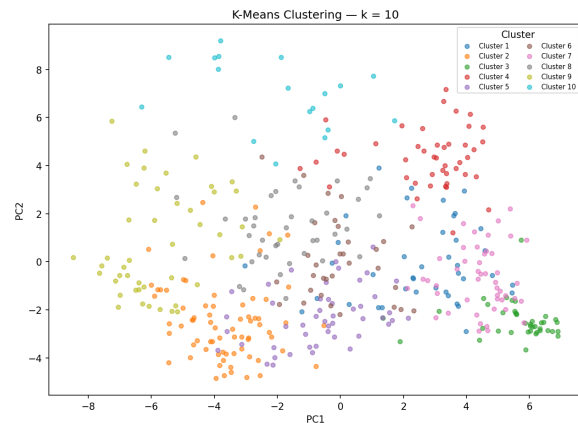


Figure 8 –  $K$ -means,  $K = 10$ : finer-grained semantic domains begin to emerge.

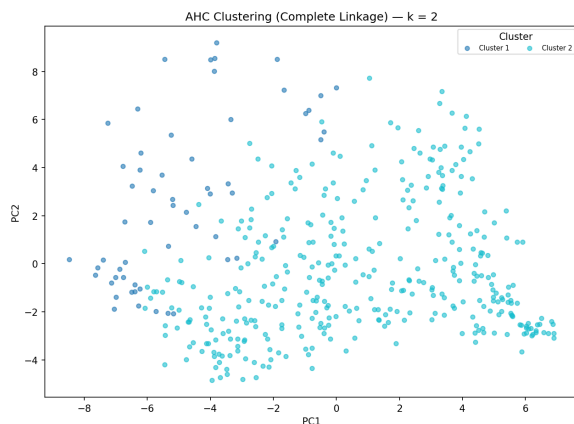


Figure 9 – AHC,  $K = 2$ .

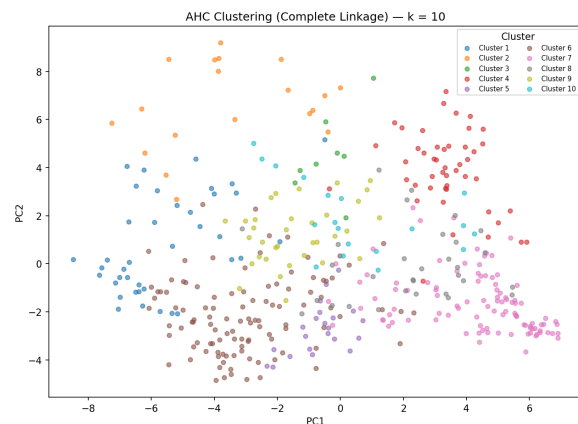


Figure 10 – AHC,  $K = 10$ : qualitatively more coherent semantic groupings than  $K$ -means (e.g. animals, food, negative affect).

## Comparison with known labels

To assess the degree of correspondence between the data-driven cluster structure and the three morphosyntactic classes (*Thing*, *Action*, *Property*), the cluster assignments at  $K = 3$  were compared with the ground-truth labels using two external validity indices: the Adjusted Rand Index (ARI) and the Normalised Mutual Information (NMI). These metrics are not introduced in (James et al., 2013) but are standard in the clustering evaluation literature; ARI equals 1 for a perfect match and is 0 in expectation for a random partition.

Table 5 reports ARI and NMI for both methods at  $K = 3$ .

## Interpretation

The unsupervised analysis yields three main findings.

First, the data-driven cluster structure does *not* align strongly with the morphosyntactic partition. This is consistent with the intrinsic difficulty of the task: the three word classes are not cleanly separable in the

Table 5 – External validity of cluster assignments at  $K = 3$  compared with the three morphosyntactic labels. Values to be filled after running `phase2_unsupervised.py` and reading from `outputs/II/phase2_unsupervised/phase2_summary.json`.

| Method                 | ARI | NMI |
|------------------------|-----|-----|
| $K$ -means ( $K = 3$ ) | -   | -   |
| AHC ( $K = 3$ )        | -   | -   |

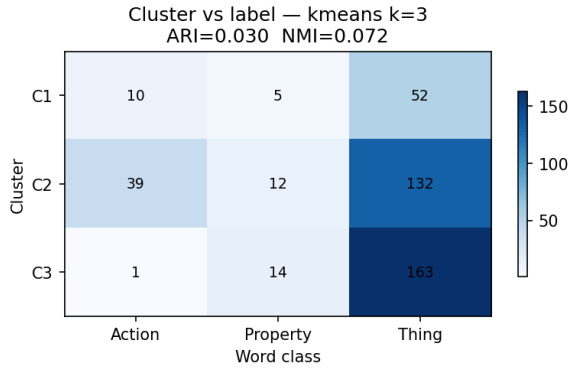


Figure 11 – Cluster-label correspondence heatmap,  $K$ -means  $K = 3$ .

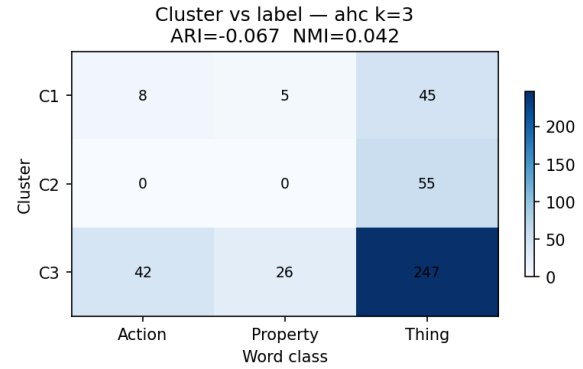


Figure 12 – Cluster-label correspondence heatmap, AHC  $K = 3$ .

semantic feature space, and the dataset is strongly imbalanced (nouns account for approximately 81% of the training set).

Second, at  $K = 2$ , both methods identify a broad **concrete vs. abstract** dimension that cuts across morphosyntactic boundaries. The concrete cluster is dominated by nouns referring to physical entities (e.g. *elephant*, *bread*), but also contains action verbs denoting physical events. The abstract cluster groups most verbs and adjectives together with nouns referring to mental or social concepts. This concrete–abstract axis corresponds to the first principal component and is the most prominent structure in the feature space.

Third, at  $K = 10$ , AHC with complete linkage recovers several semantically coherent groups — including clusters of animal names, food items, sounds, and negatively valenced concepts — that correspond to fine-grained *semantic categories* rather than to the grammatical classes of interest. This confirms that the 65-dimensional feature space encodes rich semantic similarity information at a level of granularity finer than the noun/verb/adjective distinction, and motivates the use of a supervised approach for the classification task.

These results serve a diagnostic function in the overall pipeline: they confirm the semantic informativeness of the feature space while showing that the class labels are not trivially recoverable from unsupervised exploration alone.

### 3.3 III. Supervised classification on the original feature space

In this phase, we formulate a multiclass supervised learning problem aimed at predicting the lexical-semantic labels (*Thing*, *Action*, *Property*) from the original semantic feature space. The objective is to quantify how much of the categorical structure of lexical meaning is captured by the experiential attributes and to understand the role of redundancy, multicollinearity, and model complexity in this setting.

**Baseline model** We begin with a multinomial logistic regression model, which provides a natural baseline within the generalized linear model framework. This model assumes a linear relationship between the predictors and the log-odds of class membership and serves as a reference point for all subsequent analyses.

The baseline achieves strong performance, as reported in Table 6.

| Model                         | Accuracy | Macro F1 |
|-------------------------------|----------|----------|
| Multinomial Logistic baseline | 0.9533   | 0.8904   |

Table 6 – Baseline multinomial logistic regression performance on the original labels.

The high accuracy indicates that the semantic feature space is highly informative. However, the lower macro F1 (0.8904) compared to accuracy suggests an imbalance in class-wise performance, meaning that some classes are easier to predict than others.

This behavior is confirmed by the confusion matrix in Figure 13.

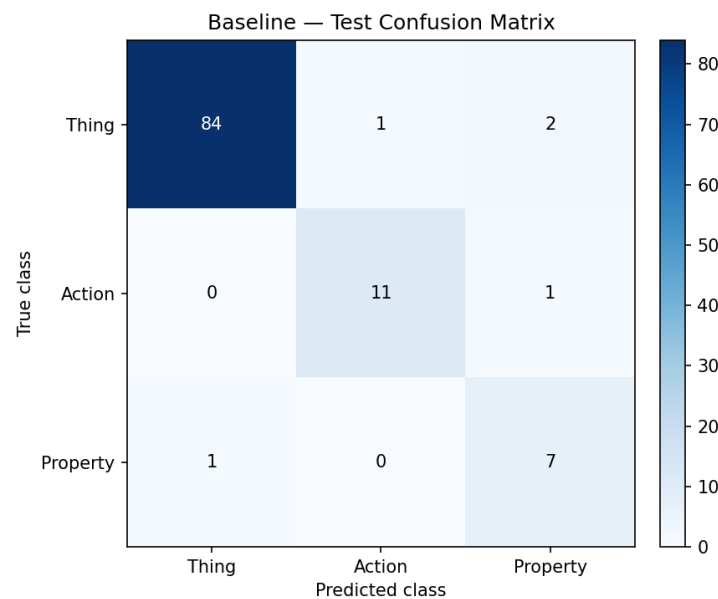


Figure 13 – Confusion matrix of the baseline multinomial logistic regression.

From the confusion matrix, it is evident that misclassifications are not uniformly distributed across classes, indicating overlapping regions in the feature space.

**Multicollinearity and feature structure** Before applying more advanced models, we analyze the structure of the feature space. The correlation heatmap in Figure 14 shows the presence of strongly correlated blocks of variables.





**Regularized models** To address multicollinearity and improve generalization, we apply LASSO and Elastic Net regularized multinomial logistic regression models.

| Model                | Accuracy | Macro F1 |
|----------------------|----------|----------|
| LASSO Logistic       | 0.9680   | 0.9150   |
| Elastic Net Logistic | 0.9705   | 0.9200   |

Table 7 – Performance of regularized multinomial models.

Both models improve over the baseline, with Elastic Net achieving the best balance between accuracy and macro F1. The improvement in macro F1 (from 0.8904 to 0.9200) is particularly important, as it indicates a more balanced classification across all classes.

**Sparsity and feature selection** Regularization also induces sparsity, reducing the number of active predictors. The most relevant features selected by LASSO are shown in Figure 16.

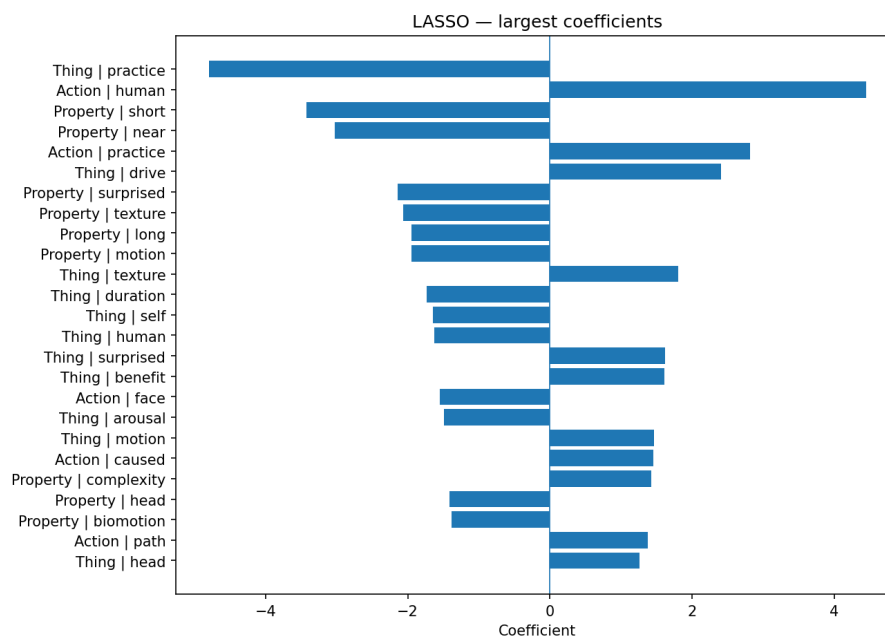


Figure 16 – Top LASSO coefficients.

Elastic Net, shown in Figure 17, distributes weights more smoothly across correlated variables.

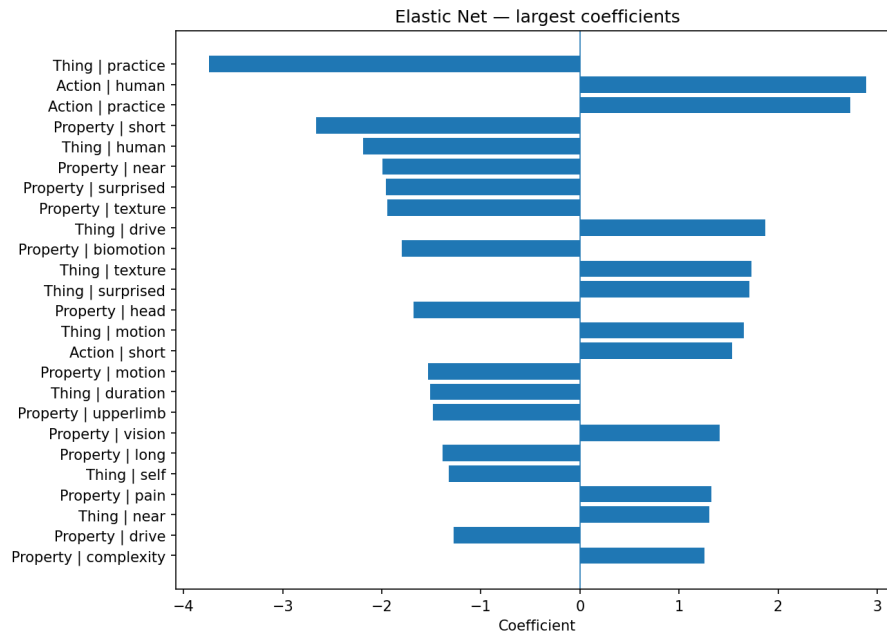


Figure 17 – Top Elastic Net coefficients.

The sparsity overview in Figure 18 shows how many coefficients are effectively used by the models.

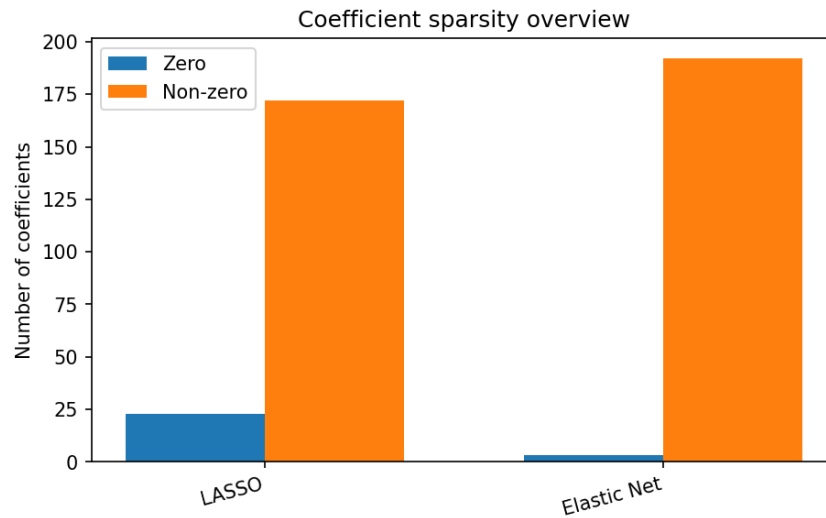


Figure 18 – Sparsity patterns across models.

These results indicate that only a subset of the semantic features is necessary to achieve high predictive performance, confirming the presence of redundancy in the original representation.

**Final post-selection model** After feature selection, we train a final multinomial logistic model on the selected subset.

| Model                   | Accuracy | Macro F1 |
|-------------------------|----------|----------|
| Post-selection Logistic | 0.9720   | 0.9229   |

Table 8 – Final post-selection model.

The improvement from 0.9533 to 0.9720 in accuracy and from 0.8904 to 0.9229 in macro F1 demonstrates that removing redundant features improves generalization.

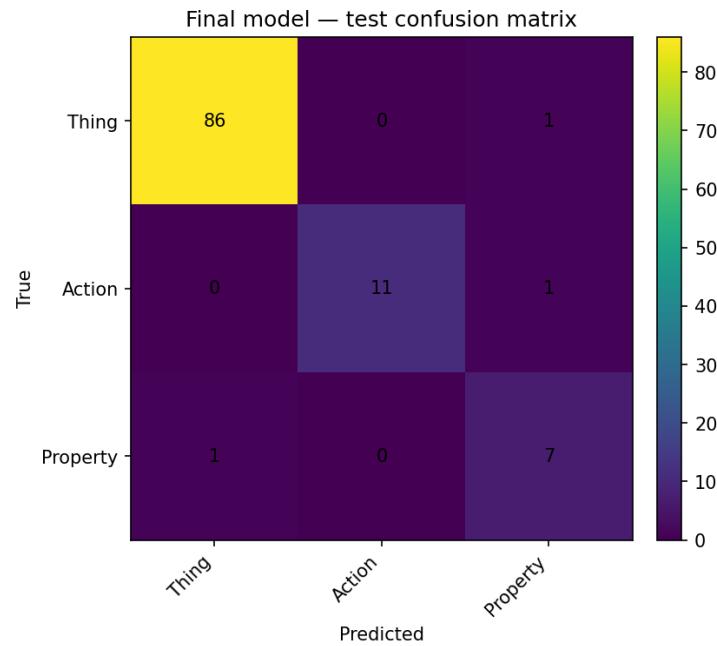


Figure 19 – Confusion matrix of the final model.

**Discussion** Overall, the results show that:

- the semantic feature space is highly informative;
- multicollinearity is substantial and must be controlled;
- regularization improves both stability and performance;
- a reduced subset of features is sufficient for accurate classification.

### 3.4 IV. High-dimensional expansion and feature screening

To further explore the structure of the data, we consider a high-dimensional extension of the feature space. The goal is to increase model expressiveness while controlling for noise and redundancy through feature screening.

**Feature screening** We rank features using ANOVA F-score and mutual information, selecting the most informative subset.

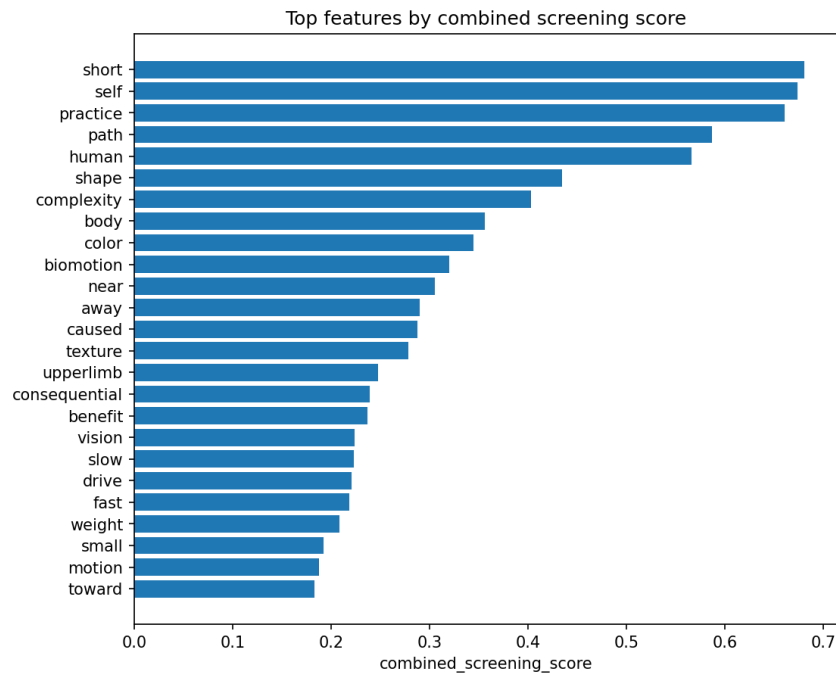


Figure 20 – Top features ranked by the combined screening score.

The ranking shows a steep decay, indicating that a limited number of features carries most of the predictive signal.

**Impact** Screening reduces dimensionality and stabilizes subsequent models, acting as a bridge between raw feature expansion and regularized learning.

### 3.5 V. Alternative classifiers

Finally, we evaluate whether non-linear models can better capture the structure of the data.

| Model                   | Accuracy | Macro F1 |
|-------------------------|----------|----------|
| Post-selection Logistic | 0.9720   | 0.9229   |
| Random Forest           | 0.9813   | 0.9486   |

Table 9 – Comparison between linear and non-linear models.

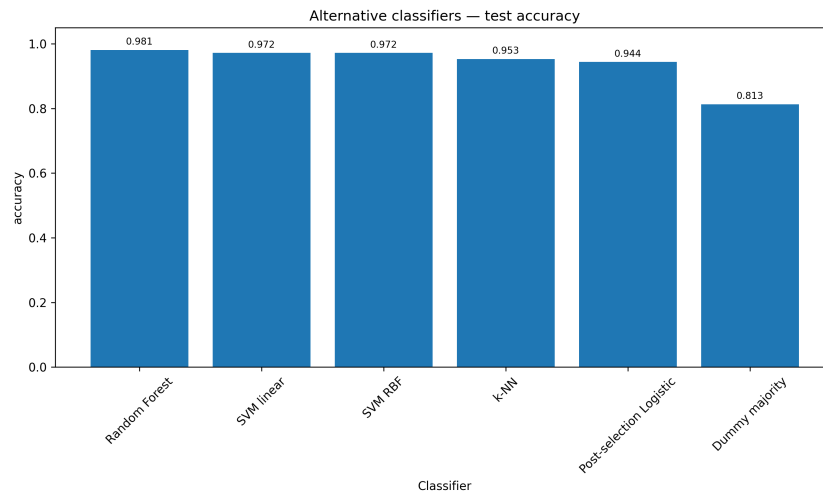


Figure 21 – Accuracy comparison across models.

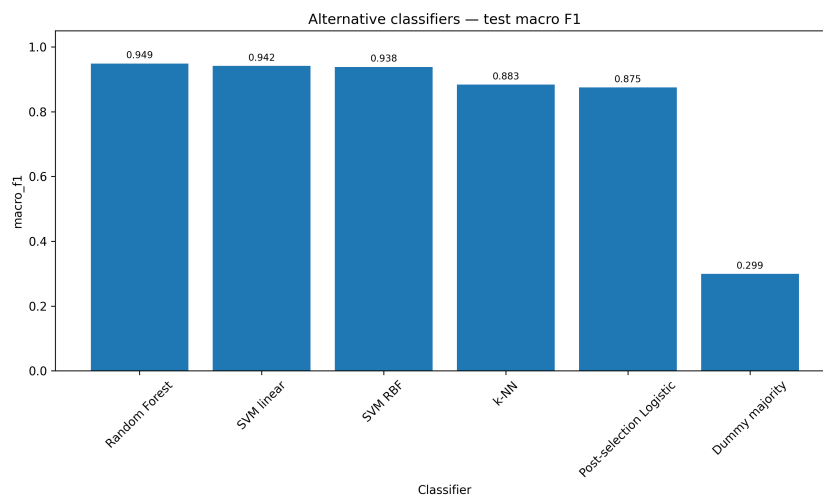


Figure 22 – Macro F1 comparison across models.

Random Forest achieves the best performance, improving macro F1 from 0.9229 to 0.9486. This indicates that non-linear interactions between features play a significant role.

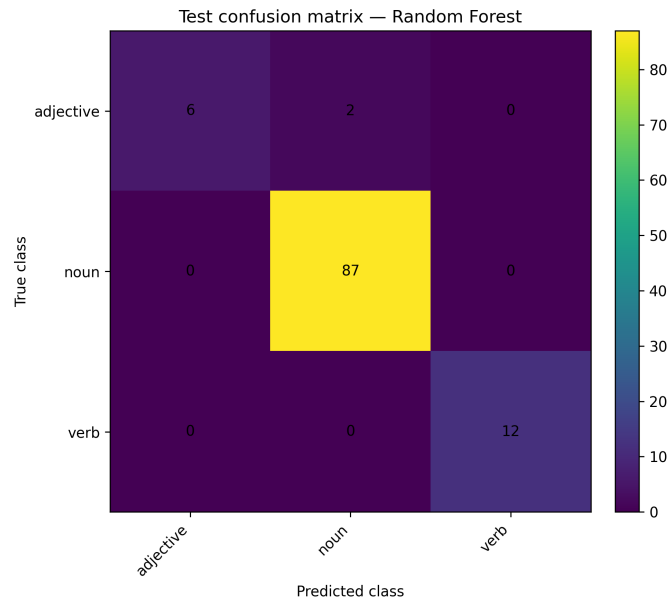


Figure 23 – Random Forest confusion matrix.

**Conclusion** The comparison highlights a key result: while linear models capture most of the signal, non-linear classifiers further improve performance by modeling interactions between features. This confirms that semantic representations are structured and that meaning emerges from combinations of attributes rather than isolated dimensions.

### 3.6 VI. Final model comparison and interpretation

This final phase integrates the results obtained across all previous analyses, with the objective of identifying the models that provide the best trade-off between predictive performance, statistical stability, and interpretability. The comparison includes the baseline multinomial logistic regression, the regularized models (LASSO and Elastic Net), the post-selection logistic model, and the alternative non-linear classifiers. From a purely predictive perspective, the results clearly indicate that performance improves progressively along the pipeline. The baseline multinomial logistic model achieves an accuracy of 0.9533 and a macro F1-score of 0.8904. Regularization leads to consistent improvements, with Elastic Net reaching 0.9705 accuracy and 0.9200 macro F1. The post-selection model further refines these results, achieving 0.9720 accuracy and 0.9229 macro F1. Finally, the Random Forest classifier outperforms all linear approaches, reaching 0.9813 accuracy and 0.9486 macro F1. These improvements are not marginal. The increase in macro F1 from 0.8904 to 0.9486 represents a substantial gain in balanced classification performance across all classes. This confirms that the successive methodological steps—regularization, feature selection, and non-linear modelling—are not only theoretically justified, but also empirically effective. However, predictive performance alone is not sufficient to determine the best model. From the perspective of interpretability, multinomial logistic regression models remain superior. The LASSO and Elastic Net solutions provide sparse and stable representations of the semantic space, identifying a reduced subset of predictors that can be directly interpreted in terms of semantic dimensions. In contrast, Random Forest offers higher predictive accuracy but at the cost of reduced interpretability, since the decision process is distributed across many trees and interactions. From a statistical learning perspective, the results also confirm the importance of controlling multicollinearity and dimensionality. The strong correlations observed among semantic features (22 pairs with  $|r| \geq 0.8$  and 51 predictors with  $VIF \geq 5$ ) make unregularized models unstable. Regularization and feature selection mitigate this issue by reducing variance and focusing the model on the most informative predictors. Overall, the comparison suggests

that no single model is universally optimal. Instead, the choice depends on the objective of the analysis. If the goal is interpretability and semantic insight, regularized multinomial models are preferable. If the goal is predictive accuracy, non-linear ensemble methods such as Random Forest provide the best results.

## 4 Results

The empirical results of the project provide a coherent picture of the statistical properties of the Binder semantic feature space and its suitability for lexical-semantic classification. First, the preprocessing phase confirms that the dataset is relatively clean but exhibits two important characteristics: moderate missingness and strong class imbalance. Missing values are concentrated in a small number of features and are successfully handled through median imputation, without loss of observations. The class distribution is highly skewed, with approximately 81% of observations belonging to the class *Thing*. This imbalance has direct consequences for model evaluation and motivates the use of metrics beyond simple accuracy. Second, the unsupervised analysis reveals that the semantic feature space contains meaningful structure, but that this structure does not align directly with the target classes. PCA shows that the first two principal components explain approximately 36% of the total variance, with a clear interpretation in terms of a concreteness–abstractness axis. Clustering methods identify coherent semantic groupings at a fine-grained level, but these clusters do not correspond strongly to the three lexical-semantic categories. This indicates that the semantic representation captures rich information, but not in a way that directly mirrors morphosyntactic classes. Third, the supervised analysis demonstrates that the semantic features are highly informative for classification. Even the baseline multinomial logistic regression achieves high accuracy (0.9533), significantly above the trivial majority-class baseline (0.811). However, the gap between accuracy and macro F1 reveals that performance is uneven across classes, with minority classes being harder to predict. The introduction of regularization leads to consistent improvements. LASSO and Elastic Net increase both accuracy and macro F1, with Elastic Net providing the best balance. These results confirm that controlling multicollinearity improves generalization. Feature selection further enhances performance, showing that a reduced subset of predictors is sufficient to capture most of the relevant signal. The high-dimensional expansion combined with feature screening demonstrates that increasing model complexity must be carefully controlled. While the expanded feature space allows the inclusion of non-linear effects and interactions, it also introduces noise and redundancy. Screening successfully reduces dimensionality, preserving the most informative predictors and stabilizing the subsequent models. Finally, the comparison with alternative classifiers shows that non-linear models can further improve performance. Random Forest achieves the best results, with 0.9813 accuracy and 0.9486 macro F1, indicating that interactions between semantic features play a significant role in the classification task. Overall, the results confirm that the Binder semantic representation is both informative and structured, but also redundant and partially non-linear. The combination of regularization, feature selection, and non-linear modelling provides the most effective strategy for exploiting this representation.

## 5 Discussion

The findings of this project provide several insights into the nature of semantic representations and the challenges of statistical learning in this domain. A first important observation is that semantic similarity and lexical classification are not equivalent problems. The unsupervised analysis shows that the feature space naturally organizes words into semantically coherent clusters, such as groups of animals, food items, or emotional concepts. However, these clusters do not align neatly with morphosyntactic categories. This suggests that lexical classes such as *Thing*, *Action*, and *Property* are not simply reflections of the geometry of the semantic space, but rather more abstract linguistic constructs. A second key result concerns the role of multicollinearity. The strong correlations observed among semantic features are not



a technical artifact, but a fundamental property of the representation. Many experiential dimensions are conceptually related and tend to co-occur. From a statistical perspective, this creates instability in model estimation and makes feature selection more challenging. The effectiveness of Elastic Net compared to LASSO alone highlights the importance of accounting for correlated predictors rather than selecting them independently. A third insight relates to the distributed nature of semantic information. The fact that good classification performance can be achieved using a reduced subset of features indicates that not all dimensions are equally informative. At the same time, the superiority of non-linear models suggests that the predictive signal is not concentrated in individual features, but rather in their interactions. This supports the view that word meaning emerges from combinations of semantic attributes rather than from isolated dimensions. The comparison between linear and non-linear models also illustrates a fundamental trade-off in statistical learning. Linear models offer interpretability and theoretical clarity, making them suitable for understanding which features matter. Non-linear models, on the other hand, provide higher predictive accuracy but are more difficult to interpret. In this project, both perspectives are necessary: linear models for interpretation, and non-linear models for performance. Finally, the high-dimensional extension of the feature space highlights the risks and opportunities of increasing model complexity. While adding interaction terms can potentially capture richer semantic patterns, it also increases noise and overfitting risk. The success of feature screening demonstrates that dimensionality reduction is essential in such settings, acting as a bridge between expressiveness and stability.

## 6 Conclusion

This project has reformulated the analysis of the Binder semantic dataset within the framework of high-dimensional statistical learning. Starting from a cognitively motivated representation of word meaning, we have developed a structured pipeline combining preprocessing, unsupervised exploration, supervised classification, multicollinearity diagnostics, feature screening, regularization, and model comparison. The results show that the semantic feature space is highly informative for lexical classification, even when using simple linear models. However, the presence of strong correlations and redundancy requires the use of regularization techniques to obtain stable and interpretable models. Feature selection further improves performance by identifying a reduced subset of relevant predictors. The extension to high-dimensional representations reveals that non-linear effects and interactions between semantic features play an important role. This is confirmed by the superior performance of Random Forest, which captures complex dependencies that linear models cannot fully represent. From a broader perspective, the project highlights the importance of combining different methodological tools. Unsupervised learning provides insight into the structure of the semantic space, supervised learning quantifies predictive performance, and regularization and screening ensure stability in high-dimensional settings. Together, these approaches offer a comprehensive framework for analysing semantic data. Future work could extend this analysis in several directions. Larger and more balanced datasets would allow more robust evaluation of minority classes. Alternative representations, such as distributional or contextual embeddings, could be compared with the Binder semantic features. Finally, more advanced models, including deep learning approaches, could be explored to further investigate the interaction between semantic structure and predictive performance. In conclusion, the Binder semantic representation proves to be a rich and informative basis for statistical learning. When combined with appropriate modelling strategies, it allows both accurate prediction of lexical classes and meaningful interpretation of the underlying semantic dimensions.

## References

- Binder, J. R., Conant, L. L., Humphries, C. J., Fernandino, L., Simons, S. B., Aguilar, M., and Desai, R. H. (2016). “Toward a Brain-Based Componential Semantic Representation”. In: *Cognitive Neuropsychology* 33.3–4, pp. 130–174. doi: [10.1080/02643294.2016.1147426](https://doi.org/10.1080/02643294.2016.1147426).
- James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Vol. 103. Springer Texts in Statistics. New York: Springer. ISBN: 978-1-4614-7137-0. doi: [10.1007/978-1-4614-7138-7](https://doi.org/10.1007/978-1-4614-7138-7).
- Testa, D. (2026). *Davide Testa LinkedIn Profile*. Public LinkedIn profile, accessed in April 2026. URL: <https://it.linkedin.com/in/davide-testa-70293b175>.

## A Original clustering and biclustering analyses

Testa's original project included an extensive exploratory unsupervised analysis of the Binder semantic feature space. The aim of this part of the work was not to predict lexical classes directly, but to investigate whether the 65 primitive semantic attributes were able to produce interpretable similarity structures among words. In other terms, Testa used unsupervised learning to address the question of whether lexical items that are semantically close in an intuitive sense also appear close when represented through Binder et al.'s componential semantic ratings. The first method applied was k-means clustering. This algorithm partitions the observations into  $k$  groups by minimizing the within-cluster sum of squared distances, so that words assigned to the same cluster are expected to have similar semantic profiles. The method was used as an initial exploratory baseline because it is simple, widely used, and provides a direct way to test whether the semantic feature space naturally separates into distinct groups. Before fixing the number of clusters, Testa evaluated different criteria, including within-cluster dissimilarity, the Hartigan index, and the average silhouette score. These diagnostics initially suggested  $k = 2$  as a possible solution, but the corresponding partition was too coarse and did not lead to a clear semantic interpretation. A second configuration with  $k = 10$  was therefore examined. Although this larger number of clusters began to reveal some semantic organization, the resulting groups remained relatively noisy and difficult to interpret. This indicated that the original 65-dimensional space contained meaningful information, but that k-means was not sufficiently flexible to recover stable semantic categories from it. The limitations of k-means motivated the use of agglomerative hierarchical clustering. Hierarchical clustering was chosen because, unlike k-means, it does not require a single fixed partition to be imposed from the beginning. Instead, it builds a dendrogram representing nested similarity relations among observations, which can then be cut at different levels of granularity. In Testa's original project, an agglomerative approach with complete linkage was used. Complete linkage defines the distance between two clusters according to the maximum distance between their members, and therefore tends to produce relatively compact clusters. As in the k-means analysis, the dendrogram was cut at  $k = 2$  and  $k = 10$ . The two-cluster solution again proved too generic, whereas the ten-cluster solution produced more interpretable semantic groupings. In particular, some clusters could be associated with broad semantic areas such as food, animals, natural events, musical instruments, and sentiment-related words. This result suggested that the semantic ratings did encode meaningful similarity relations, but also that the structure of the original feature space was not sharply separated into clean clusters. A further step was the application of biclustering. This method was introduced because standard clustering groups either observations or variables, whereas biclustering simultaneously clusters rows and columns of a data matrix. This is especially relevant for semantic feature data: two words may be similar only with respect to a specific subset of semantic dimensions, rather than across all 65 features. For example, a group of words may be close in auditory features but not in visual or social features. Biclustering is therefore better suited to identifying local semantic patterns, where subsets of lexical items are associated with subsets of experiential attributes. In Testa's original project, biclustering was performed using the *isa2* package in R, based on the Iterative Signature Algorithm. Among the biclustering approaches tested, this method produced the most coherent semantic output. The biclustering analysis identified a large number of local structures. Testa reported 199 biclusters, many of which were semantically interpretable both in terms of the words grouped together and in terms of the features involved. The eight most relevant or coherent biclusters were manually interpreted as corresponding to semantic domains such as *Time*, *Food and Drinks*, *Animals*, *Sounds*, *Locations*, *Negative Events and Feelings*, *Human Beings* and *Practical Tools*. This result was important because it showed that the Binder feature matrix contains local semantic regularities: groups of words are not only similar globally, but can share specific subsets of experiential dimensions. At the same time, the analysis also revealed substantial overlap among biclusters, meaning that the same lexical item could belong to multiple semantic patterns. This overlap is not necessarily a weakness; rather, it reflects the fact that word meaning is multidimensional and that a single concept can participate in different semantic domains. After these first clustering attempts, Testa introduced Principal Component Analysis as a denoising and dimensionality reduction step. The rationale was that the original semantic space is

high enough in dimension to contain redundancy, correlated predictors, and noise. PCA transforms the original variables into orthogonal linear combinations, ordered by the amount of variance they explain. By projecting the data onto a smaller number of principal components, it becomes possible to retain the dominant structure of the dataset while reducing the influence of less informative variation. In Testa's analysis, the first ten principal components were reported to explain approximately 80% of the total variance, with the first few components carrying the largest share of information. The PCA loadings were then inspected to understand the semantic meaning of the main components. In particular, the first two principal components were visualized through a correlation circle and feature contribution plots. Testa interpreted these two components as reflecting two broad and opposite latent semantic dimensions. One component was mainly associated with temporal, causal, social, emotional and attentional features, and was interpreted as closer to an abstract semantic dimension. The other component was more strongly associated with sensory-motor and spatial features, and was interpreted as closer to a concrete semantic dimension. This observation led to one of the main qualitative conclusions of the original project: the opposition between *Concreteness* and *Abstractness* may represent an important latent structure in the semantic organization of the dataset. PCA was also used as a preprocessing step before repeating the clustering analysis. The reason for doing this was that clustering in the original feature space may be affected by redundancy and multicollinearity, while clustering in a lower-dimensional PCA space may reveal cleaner groupings. Testa therefore applied k-means and hierarchical clustering to the PCA-reduced representation. Hierarchical clustering again produced the most meaningful results. With  $k = 2$ , the clusters approximately separated abstract from concrete lexical items. With larger values of  $k$ , especially between  $k = 10$  and  $k = 28$ , the clusters became more semantically specific and closer to the semantic macro-categories identified in Binder et al.'s dataset. However, this increase in semantic detail came with a reduction in the average silhouette score. Testa therefore interpreted the range between 10 and 28 clusters as a trade-off between quantitative cluster compactness and qualitative semantic interpretability. Overall, the unsupervised analyses in Testa's original project had three main purposes. First, they served as a validation of the Binder semantic feature representation, showing that the 65 experiential attributes were not arbitrary but contained recoverable similarity structures. Second, they provided exploratory evidence that some semantic domains could be identified from the data without using lexical-class labels. Third, they motivated the later supervised analysis by showing that the semantic feature space already contained structure relevant to lexical organization. Among the methods tested, simple k-means was the least successful, hierarchical clustering was more interpretable, PCA improved the clarity of the cluster structure, and biclustering provided the richest local semantic patterns. In the present work, these analyses are reported only as background and exploratory evidence. They are not part of the main methodological contribution, because the focus is shifted from unsupervised semantic grouping to supervised high-dimensional statistical learning. Nevertheless, they remain relevant because they justify the use of the Binder feature space as a meaningful input representation. Testa's original project showed that the semantic ratings contain interpretable structure; the present project starts from that result and asks a different question, namely how regularization, feature screening, rebalancing and stability analysis behave when this semantic representation is expanded into a large-feature-space classification problem.

## B Detailed explanation of the Python preprocessing pipeline

This appendix describes in detail the three Python scripts used to implement the preprocessing phase of the project. The scripts correspond to the first three operational steps of the analysis pipeline and are stored as separate modules in order to keep the workflow transparent, reproducible, and easy to inspect. The three scripts are:

- `dataset_initial_cleaning.py`, corresponding to Phase 1.1;
- `dataset_train_test_split.py`, corresponding to Phase 1.2;
- `dataset_imputation_scaling.py`, corresponding to Phase 1.3.

The overall objective of these scripts is not to fit predictive models, but to construct a reliable analytical

dataset. For this reason, the preprocessing pipeline is deliberately separated from the modelling pipeline. In particular, the scripts avoid performing operations that could introduce data leakage, such as imputing missing values or estimating scaling parameters before the train–test split. Each phase saves its own outputs inside the `outputs/I/` directory, so that the intermediate results can be examined and reused by later phases of the project.

## B.1 Phase 1.1: Initial dataset cleaning

The first script, `dataset_initial_cleaning.py`, performs the initial structural inspection and cleaning of the 65-feature semantic dataset. The input file is:

```
dataset/slld_binder_Xy_65_features.csv
```

The script saves its outputs in:

```
outputs/I/phase1_1_initial_cleaning/
```

This phase is intentionally conservative. Its purpose is not to transform the data statistically, but to verify that the dataset has the expected structure and to produce a clean raw version of the data that can be safely used in later phases.

### B.1.1 Path definition and output organization

At the beginning of the script, the input and output paths are defined using the `Path` class from Python's `pathlib` module. This makes the code more readable and more robust than relying on raw string paths.

The script defines the main dataset directory:

```
DATASET_DIR = Path("./dataset")
```

and the output directory:

```
OUTPUT_DIR = Path("./outputs/I//phase1_1_initial_cleaning")
```

The double slash in `I//phase1_1_initial_cleaning` does not change the semantic meaning of the path: operating systems normalize it automatically. The important point is that all outputs from Phase 1.1 are stored inside the folder `outputs/I/`, which identifies the first main block of the project pipeline.

The script produces five files:

- `slld_phase1_1_clean_raw.csv`, the structurally cleaned raw dataset;
- `missing_values_report.csv`, a report on missing values by feature;
- `duplicated_word_forms.csv`, a report on repeated word forms;
- `target_class_distribution.csv`, a report on class frequencies;
- `phase1_1_summary.json`, a machine-readable summary of the phase.

The presence of both CSV reports and a JSON summary is useful because the CSV files are easy to inspect manually, while the JSON file can be automatically read by later scripts or used for reproducibility checks.

### B.1.2 Expected columns and target validation

The script expects three non-feature columns:

```
entry_id, word, target_word_class
```

These columns are stored in the list:

```
ID_COLS = ["entry_id", "word", "target_word_class"]
```

All remaining columns are treated as semantic feature columns. Since the dataset is expected to contain 65 semantic predictors, the script checks whether exactly 65 feature columns are detected. If the number is different from 65, the script does not immediately stop, but prints a warning. This is a sensible choice because a mismatch in the number of predictors may indicate that the wrong input file has been loaded, but it does not necessarily mean that the file is unreadable.

The script also defines the valid target classes as:

```
{"noun", "verb", "adjective"}
```

This check is essential because the whole supervised part of the project assumes a three-class classification task. Rows whose target label is not included in this set are treated as structurally invalid. If such rows are found, the script prints them and removes them. In the prepared dataset, this step is expected to remove no observations.

### B.1.3 Loading the dataset and checking file existence

Before loading the dataset, the script checks whether the input file exists. If the file is missing, it raises a `FileNotFoundError`. This is preferable to letting `pandas` fail later with a less informative error message.

The dataset is then loaded using:

```
pd.read_csv(INPUT_FILE)
```

After loading, the script prints the number of rows and columns. This immediate diagnostic is useful because it allows us to verify that the expected dataset has been loaded before any cleaning operation is applied.

### B.1.4 Textual cleaning of word and target columns

The script performs a minimal cleaning of the textual columns `word` and `target_word_class`. Specifically, it converts the values to strings and removes accidental leading or trailing whitespace:

```
astype(str).str.strip()
```

This operation is deliberately limited. The script does not lowercase the words, does not lemmatize them, and does not modify their lexical content. This is important because the lexical item is part of the dataset identity. The preprocessing phase should not alter linguistic information unless there is a clear methodological justification.

### B.1.5 Exact duplicate rows

The script then checks for exact duplicate rows using:

```
df.duplicated().sum()
```

Exact duplicates are rows that are identical across all columns. If such rows are present, they are removed with:

```
df.drop_duplicates()
```

This choice is methodologically safe because exact duplicates do not introduce new information. Keeping them would artificially increase the weight of repeated observations in later analyses.

It is important to distinguish exact duplicate rows from duplicated word forms. Exact duplicate rows are redundant observations; duplicated word forms may instead correspond to legitimate lexical entries.

### B.1.6 Duplicated word forms

After removing exact duplicates, the script searches for duplicated values in the `word` column:

```
df_clean.duplicated("word", keep=False)
```

The result is sorted by word and target class and saved to:

```
duplicated_word_forms.csv
```

Duplicated word forms are reported but not removed. This is a crucial methodological decision. The statistical unit of this project is not simply the written word form, but the lexical entry. Therefore, the same orthographic word can be validly represented more than once if it corresponds to different lexical-semantic roles. For instance, a word may occur once as a verb and once as an adjective. Removing one of these entries would incorrectly collapse distinct lexical uses.

### B.1.7 Missing-value analysis

The script computes missing values only on the feature columns, not on the identifier columns. For each semantic feature, it calculates:

- the number of missing values;
- the percentage of missing values.

The missing-value report is sorted in descending order, so that the most problematic features appear first. The report is saved as:

```
missing_values_report.csv
```

The script also computes:

- the number of rows with at least one missing value;

— the number of complete rows.

This information is important because it quantifies the cost of complete-case analysis. If rows with missing values were removed immediately, the sample size would decrease. Since the dataset is small and class-imbalanced, aggressive deletion would be statistically undesirable.

For this reason, Phase 1.1 does not impute missing values and does not remove incomplete rows. Missing values are only described and preserved for later treatment.

### B.1.8 Class distribution

The script computes the distribution of `target_word_class` using:

```
value_counts()
```

It then converts the counts into percentages and saves the output to:

```
target_class_distribution.csv
```

This step is essential because the target distribution determines how model performance must be interpreted. If one class is much more frequent than the others, raw accuracy can be misleading. A classifier can obtain high accuracy simply by predicting the majority class. Therefore, the class distribution report produced in Phase 1.1 motivates the use of balanced accuracy, macro-F1, class-wise recall, and confusion matrices in later phases.

### B.1.9 Saving the clean raw dataset

At the end of the script, the cleaned dataset is saved as:

```
s1ld_phase1_1_clean_raw.csv
```

This file is called *clean raw* because it is structurally cleaned but not statistically transformed. In particular, the script explicitly avoids:

- removing rows with missing values;
- removing columns with missing values;
- imputing missing values;
- standardizing features;
- performing a train–test split.

This design is methodologically correct because imputation and scaling must be performed after the train–test split, and they must be fitted only on the training set.

### B.1.10 Summary JSON

Finally, the script saves a JSON file containing the main numerical information and methodological decisions of Phase 1.1. The JSON summary includes:

- input and output file paths;
- initial and cleaned dataset dimensions;
- number of identifier columns and feature columns;



- valid target classes;
- number of exact duplicates removed;
- number of duplicated word-form rows;
- number of rows with missing values;
- list of columns with missing values;
- explicit methodological decisions.

This JSON file makes the preprocessing phase auditable. Instead of relying only on the printed console output, the project stores a structured summary that can be inspected later.

### B.1.11 Pseudocode for Phase 1.1

---

**Algorithm 1** Phase 1.1: Initial dataset cleaning
 

---

```

1: Define input and output paths
2: Create output directory if it does not exist
3: Load the 65-feature dataset
4: Check that required columns are present
5: Identify feature columns as all non-identifier columns
6: if number of feature columns is not 65 then
7:   Print warning
8: end if
9: Strip accidental whitespace from word and target_word_class
10: Check that all target labels are valid
11: if invalid target labels are found then
12:   Remove invalid rows
13: end if
14: Count exact duplicate rows
15: Remove exact duplicate rows
16: Identify duplicated word forms
17: Save duplicated word-form report
18: Compute missing-value counts and percentages by feature
19: Save missing-value report
20: Compute target class distribution
21: Save target class distribution report
22: Save structurally cleaned raw dataset
23: Save JSON summary of the phase
  
```

---

## B.2 Phase 1.2: Stratified train–test split

The second script, `dataset_train_test_split.py`, loads the structurally cleaned raw dataset produced in Phase 1.1 and creates a reproducible train–test split. Its input file is:

`outputs/I/phase1_1_initial_cleaning/sl1d_phase1_1_clean_raw.csv`

The outputs are saved in:

`outputs/I/phase1_2_train_test_split/`

This phase is crucial because all later preprocessing operations must be fitted only on the training set. Therefore, the split is performed before imputation and before standardization.

### B.2.1 Purpose of the split

The script separates the data into:

- a training set, used to estimate preprocessing parameters and fit models;
- a test set, used only for external evaluation.

The split is stratified by the target variable `target_word_class`. Stratification is necessary because the dataset is strongly class-imbalanced. Without stratification, the random split could allocate too few verbs or adjectives to the test set, making evaluation unstable and potentially misleading.

### B.2.2 Configuration

The script defines the following main configuration parameters:

- `TEST_SIZE = 0.20`, meaning that 20% of the observations are assigned to the test set;
- `RANDOM_STATE = 42`, ensuring reproducibility;
- `TARGET_COL = "target_word_class"`, defining the stratification variable.

The use of a fixed random state is important because it guarantees that the same train–test split can be reproduced in future runs. Reproducibility is particularly important in a small and imbalanced dataset, where different splits could lead to noticeably different model performance.

### B.2.3 Helper function for class distribution

The function:

```
compute_class_distribution(df, split_name)
```

computes, for a given dataset split:

- the count of each class;
- the percentage of each class;
- the name of the split to which the report refers.

The function returns a pandas DataFrame. This design avoids repeated code and ensures that the class distributions for the full dataset, training set, and test set are computed in the same way.

### B.2.4 Helper function for missing values by split

The function:

```
compute_missing_summary(df, feature_cols, split_name)
```

computes missing-value counts and percentages for each feature within a given split. This is important because missingness may not be distributed identically between the training and test sets. By saving missing-value reports after the split, the project can verify how many missing values will have to be handled in each subset.

### B.2.5 Structural checks

After loading the cleaned raw dataset, the script repeats some structural checks:

- it verifies that the identifier columns are present;
- it checks that all target labels are valid;
- it identifies the feature columns;
- it checks whether 65 feature columns are detected.

Repeating these checks is not redundant. Since Phase 1.2 depends on the output of Phase 1.1, these checks ensure that the correct file has been loaded and that the file has not been modified accidentally.

### B.2.6 Minimum class-count check

Before performing the stratified split, the script calculates the minimum class count:

```
df[TARGET_COL].value_counts().min()
```

If any class has fewer than two observations, a stratified train–test split would not be feasible. The script therefore raises an error in this case. This is a defensive programming choice: it prevents the procedure from silently producing an invalid split.

### B.2.7 Stratified train–test split

The split is performed using `train_test_split` from `sklearn.model_selection`. The key argument is:

```
stratify=df[TARGET_COL]
```

This ensures that the relative class frequencies in the training and test sets remain close to those in the full dataset.

The script uses:

```
shuffle=True
```

so that observations are randomly assigned to train and test while preserving class proportions.

After splitting, both datasets are sorted by `entry_id`. This sorting does not affect the statistical content of the split, but improves readability and makes saved files easier to inspect.

### B.2.8 Saving raw train and test datasets

The script saves:

- `s1ld_phase1_2_train_raw.csv`;
- `s1ld_phase1_2_test_raw.csv`.

These files are still raw with respect to missing values and scaling. They are separated into train and test, but no imputation and no standardization have been performed. This is the correct order of operations because the imputer and scaler will later be fitted only on the training set.

### B.2.9 Class distribution after splitting

The script computes class distributions for:

- the full dataset;
- the training set;
- the test set.

These reports are concatenated and saved in:

`split_class_distribution.csv`

This file allows us to verify empirically that stratification worked as intended. The expected outcome is that the percentage of nouns, verbs, and adjectives remains approximately the same across the three splits.

### B.2.10 Majority-class baseline

The script also computes the majority-class baseline. It identifies the most frequent class in the full dataset and calculates the accuracy that would be obtained by always predicting that class.

This value is saved in the summary JSON under:

`majority_class_baseline`

The majority baseline is important because it gives a minimal benchmark for later models. In an imbalanced classification task, a model must not be considered successful simply because it achieves high accuracy. It must improve over the majority baseline and, more importantly, it must perform adequately on minority classes.

### B.2.11 Missing values by split

The script computes missing-value summaries for the full dataset, training set, and test set. These summaries are concatenated and saved as:

`missing_values_by_split.csv`

This step prepares Phase 1.3. By knowing which features contain missing values in the training and test sets, we can later verify whether imputation has removed all missing entries.

### B.2.12 Summary JSON

The Phase 1.2 JSON summary records:

- input and output paths;
- number of rows in the full dataset, training set, and test set;
- number of columns and feature columns;
- test size and random state;
- stratification column;
- majority-class baseline;
- class distributions;

- missing-value counts by split;
- methodological decisions.

The most important methodological decision recorded in this phase is that imputation, standardization, and feature selection are not performed during splitting. They are explicitly postponed to later phases.

### B.2.13 Pseudocode for Phase 1.2

---

#### **Algorithm 2** Phase 1.2: Stratified train–test split

---

- 1: Define input and output paths
  - 2: Create output directory if it does not exist
  - 3: Load the cleaned raw dataset from Phase 1.1
  - 4: Check required columns and valid target classes
  - 5: Identify feature columns
  - 6: Compute class distribution of full dataset
  - 7: Check that each class has enough observations for stratification
  - 8: Split data into train and test using stratification by target class
  - 9: Sort train and test sets by `entry_id`
  - 10: Save raw training dataset
  - 11: Save raw test dataset
  - 12: Compute class distributions for full, train, and test sets
  - 13: Save class-distribution report
  - 14: Compute majority-class baseline accuracy
  - 15: Compute missing-value summaries by split
  - 16: Save missing-value report by split
  - 17: Save JSON summary of the phase
- 

### B.3 Phase 1.3: Imputation and standardization

The third script, `dataset_imputation_scaling.py`, performs missing-value imputation and feature standardization. It uses as input the raw training and test files produced by Phase 1.2:

`s1ld_phase1_2_train_raw.csv`

and

`s1ld_phase1_2_test_raw.csv`

The outputs are saved in:

`outputs/I/phase1_3_imputation_scaling/`

This script is the first phase in which statistical transformations are fitted. For this reason, it is the most important preprocessing phase from the point of view of leakage control.

### B.3.1 Input validation

The script first checks that both train and test input files exist. If either file is missing, it raises a `FileNotFoundError` and instructs the user to run Phase 1.2 first.

After loading the datasets, the script verifies that the required identifier columns are present in both train and test. It then identifies feature columns separately in the two datasets and checks that they match exactly. This is important because train and test must have the same predictors in the same order before imputation and scaling are applied.

### B.3.2 Feature-column conversion to numeric

Before imputation, all feature columns are converted to numeric values using:

```
pd.to_numeric(..., errors="coerce")
```

This operation ensures that the semantic predictors are treated as numerical variables. If a non-numeric value is accidentally present in a feature column, it is converted to `NaN`. This is useful because the imputation step can then handle it consistently as a missing value.

The choice `errors="coerce"` is defensive: it prevents the script from failing on isolated malformed numerical entries and instead includes them in the missing-value treatment.

### B.3.3 Missing values before imputation

The script defines a helper function:

```
compute_missing_report(df, feature_cols, split_name, stage)
```

This function computes the missing-value count and percentage for each feature. It also records both the split name and the stage of processing, for example:

```
— train, before_imputation;  
— test, before_imputation;  
— train, after_imputation;  
— test, after_imputation.
```

This design makes it possible to combine all missing-value diagnostics in a single file and directly compare the situation before and after imputation.

The script also computes the number of rows with at least one missing value in both train and test. This provides a row-level measure of missingness, complementing the feature-level report.

### B.3.4 Median imputation fitted on training data

The script uses:

```
SimpleImputer(strategy="median")
```

from `sklearn.impute`. Median imputation is chosen because it is simple, robust, and appropriate for numerical semantic ratings. Compared with mean imputation, it is less sensitive to skewed distributions or outlying values.

The most important methodological point is that the imputer is fitted only on the training feature matrix:

```
imputer.fit_transform(train_df[feature_cols])
```

The test set is then transformed using the already fitted imputer:

```
imputer.transform(test_df[feature_cols])
```

This distinction between `fit_transform` on train and `transform` on test is essential. The median values used for imputation must be estimated without looking at the test set. If test-set values were used to compute imputation medians, information from the evaluation data would enter the preprocessing pipeline, creating data leakage.

### B.3.5 Reconstructing imputed datasets

The imputer returns NumPy arrays. The script converts these arrays back into pandas DataFrames using the original feature names and row indices. It then concatenates the identifier columns with the imputed features.

This produces two complete datasets:

```
— train_imputed_df;  
— test_imputed_df.
```

Both retain the original columns `entry_id`, `word`, and `target_word_class`, followed by the 65 imputed semantic features.

### B.3.6 Saving imputation values

The script saves the imputation statistics in:

```
imputation_values.csv
```

For each feature, this file reports:

- the feature name;
- the imputation strategy;
- the median value fitted on the training set;
- the number of missing values in the training set before imputation;
- the number of missing values in the test set before imputation.

This file is important for reproducibility and interpretability. It allows us to know exactly which value was used to replace missing entries in each feature.

### B.3.7 Missing values after imputation

After imputation, the script recomputes missing-value reports for train and test. It concatenates the before-imputation and after-imputation reports and saves them as:

```
missing_values_before_after.csv
```

The expected result is that both train and test contain zero missing values after imputation. This is verified both feature by feature and at the row level.

### B.3.8 Saving imputed but unscaled datasets

The script saves the imputed datasets before scaling:

- `sllld_phase1_3_train_imputed.csv`;
- `sllld_phase1_3_test_imputed.csv`.

These files are useful because not all later methods require standardized predictors. For instance, tree-based models such as Random Forests do not require feature scaling. Keeping the imputed but unscaled version of the data avoids forcing all later analyses to use standardized variables.

### B.3.9 Standardization fitted on training data

The second transformation performed by the script is standardization. The script uses:

`StandardScaler()`

from `sklearn.preprocessing`. Standardization transforms each feature according to:

$$z_{ij} = \frac{x_{ij} - \hat{\mu}_{j,\text{train}}}{\hat{\sigma}_{j,\text{train}}},$$

where  $\hat{\mu}_{j,\text{train}}$  and  $\hat{\sigma}_{j,\text{train}}$  are the mean and standard deviation of feature  $j$  computed on the training set only.

As with imputation, the scaler is fitted only on the training set:

`scaler.fit_transform(train_imputed_df[feature_cols])`

The test set is transformed using the training-set scaling parameters:

`scaler.transform(test_imputed_df[feature_cols])`

This is necessary to preserve the test set as external data. The test set must not be centered and scaled using its own mean and standard deviation, because doing so would allow information from the test distribution to influence the preprocessing.

### B.3.10 Why standardization is necessary

Standardization is required because many later statistical learning procedures are scale-sensitive. In particular:

- PCA depends on variances and covariances;
- clustering depends on distances between observations;
- k-nearest neighbors depends directly on the metric structure of the feature space;
- Ridge, LASSO, and Elastic Net penalize coefficients, and the penalty is not comparable across variables measured on different scales.

Without standardization, features with larger numerical dispersion could dominate the analysis, not because they are more semantically important, but because they are measured on a larger scale.



### B.3.11 Saving scaling parameters

The script saves the mean and standard deviation used for each feature in:

`scaling_parameters.csv`

For each feature, the file reports:

- the feature name;
- the training-set mean used for scaling;
- the training-set standard deviation used for scaling.

This makes the standardization fully reproducible.

### B.3.12 Saving standardized datasets

The standardized datasets are saved as:

- `s1ld_phase1_3_train_scaled.csv`;
- `s1ld_phase1_3_test_scaled.csv`.

These are the main inputs for PCA, clustering, distance-based models, and regularized regression/classification models.

### B.3.13 Diagnostics on standardized training data

After scaling, the script computes diagnostic quantities on the standardized training matrix:

- the maximum absolute feature mean;
- the minimum feature standard deviation;
- the maximum feature standard deviation.

Since the scaler is fitted on the training set, the training features should have means close to zero and standard deviations close to one. Small deviations from exact zero or one are expected only because of floating-point numerical precision.

These diagnostics are saved in the summary JSON and printed to the console.

### B.3.14 Summary JSON

The Phase 1.3 JSON summary records:

- input and output paths;
- train and test dimensions;
- number of feature columns;
- imputation strategy;
- confirmation that imputation is fitted only on the training set;
- standardization method;
- confirmation that standardization is fitted only on the training set;
- missing-value counts before and after imputation;
- standardization diagnostics;
- methodological decisions.

The summary explicitly states that PCA, clustering, feature selection, and model training are not performed in this phase. This reinforces the modular structure of the pipeline: Phase 1.3 prepares the data, while later phases analyze and model them.

### B.3.15 Pseudocode for Phase 1.3

---

**Algorithm 3** Phase 1.3: Imputation and standardization
 

---

```

1: Define input and output paths
2: Create output directory if it does not exist
3: Load raw training and test datasets from Phase 1.2
4: Check required columns in train and test sets
5: Identify feature columns in train and test
6: if train and test feature columns do not match then
7:   Stop with an error
8: end if
9: Convert all feature columns to numeric values
10: Compute missing-value reports before imputation
11: Count rows with at least one missing value in train and test
12: Initialize median imputer
13: Fit imputer on training features only
14: Transform training features
15: Transform test features using training-fitted imputer
16: Reconstruct imputed train and test datasets
17: Save imputation values
18: Compute missing-value reports after imputation
19: Save before/after missing-value report
20: Save imputed but unscaled train and test datasets
21: Initialize standard scaler
22: Fit scaler on imputed training features only
23: Transform imputed training features
24: Transform imputed test features using training-fitted scaler
25: Reconstruct standardized train and test datasets
26: Save scaling parameters
27: Save standardized train and test datasets
28: Compute standardization diagnostics on training set
29: Save JSON summary of the phase
  
```

---

## B.4 Overall logic of the preprocessing pipeline

The three scripts implement a strict separation between structural cleaning, data splitting, and statistical transformation. This order is essential for a valid supervised learning analysis.

In Phase 1.1, we only inspect and clean the dataset structurally. We remove exact duplicates, report duplicated word forms, inspect missingness, and save the class distribution. No train–test split, imputation, or scaling is performed.

In Phase 1.2, we split the structurally cleaned data into training and test sets. The split is stratified because the target classes are imbalanced. Still, no imputation or scaling is performed.

In Phase 1.3, we finally fit the imputer and scaler. Both are fitted exclusively on the training set and then applied to the test set. This guarantees that the test set does not influence preprocessing parameters.

The complete preprocessing pipeline can be summarized as follows:

**Algorithm 4** Complete preprocessing workflow

- 
- 1: Load original 65-feature semantic dataset
  - 2: Perform structural cleaning
  - 3: Save clean raw dataset
  - 4: Create stratified train–test split
  - 5: Save raw train and test datasets
  - 6: Fit imputer on train only
  - 7: Apply imputer to train and test
  - 8: Save imputed train and test datasets
  - 9: Fit scaler on imputed train only
  - 10: Apply scaler to train and test
  - 11: Save standardized train and test datasets
  - 12: Use standardized data for PCA, clustering, regularized models, and distance-based classifiers
  - 13: Use imputed unscaled data when scaling is not required
- 

This structure ensures that the preprocessing stage is reproducible, transparent, and statistically valid. It also creates multiple versions of the dataset, each suited to different later methods: raw split data for diagnostics, imputed data for models that do not require scaling, and standardized data for PCA, clustering, distance-based learning, and regularized classification.

## C Detailed explanation of the Python analysis pipeline

This section continues the documentation of the Python workflow after the preprocessing pipeline. Once the clean, imputed, and standardized datasets have been produced, the project proceeds through unsupervised exploration, supervised classification, multicollinearity diagnostics, feature screening, regularized modelling, sparsity analysis, final model evaluation, alternative classifiers, and supervised classification based on clustering-derived labels.

The scripts documented in this section are:

- `phase2_unsupervised.py`, corresponding to Phase 2;
- `phase3_1_supervised_data_loading_checks.py`, corresponding to Phase 3.1;
- `phase3_2_train_test_baseline.py`, corresponding to Phase 3.2;
- `phase3_3_multicollinearity_analysis.py`, corresponding to Phase 3.3;
- `phase3_4_feature_screening.py`, corresponding to Phase 3.4;
- `phase3_5_regularized_models.py`, corresponding to Phase 3.5;
- `phase3_6_sparsity_analysis.py`, corresponding to Phase 3.6;
- `phase3_7_final_model_evaluation.py`, corresponding to Phase 3.7;
- `phase3_extra_classifiers_comparison.py`, corresponding to the extra classifier comparison;
- `phase4_cluster_label_supervised_analysis.py`, corresponding to Phase 4.

The guiding principle remains the same as in preprocessing: each script reads explicitly defined inputs, creates a dedicated output directory, saves tabular reports and figures, and writes a JSON summary. This makes the pipeline auditable and reproducible.

## C.1 Phase 2: Unsupervised exploration of the semantic space

### C.1.1 File: `phase2_unsupervised.py`

The script `phase2_unsupervised.py` performs the unsupervised exploration of the semantic space. Its input is the standardized training dataset produced by Phase 1.3:

```
outputs/I/phase1_3_imputation_scaling/sl1d_phase1_3_train_scaled.csv
```

The outputs are saved in:

```
outputs/II/phase2_unsupervised/
```

The goal of this phase is exploratory. The script does not fit a supervised classifier and does not use the target labels for model training. Instead, it investigates whether the standardized semantic feature space contains interpretable structure. In particular, it applies Principal Component Analysis, K-Means clustering, and Agglomerative Hierarchical Clustering. The known word-class labels are used only after clustering, to evaluate how much the unsupervised structure agrees with the original lexical-semantic classes.

### C.1.2 Path definition and configuration

The script defines the input directory:

```
PHASE1_3_DIR = Path("./outputs/I/phase1_3_imputation_scaling")
```

and the output directory:

```
OUTPUT_DIR = Path("./outputs/II/phase2_unsupervised")
```

The main configuration parameters are:

- `ID_COLS`, containing `entry_id`, `word`, and `target_word_class`;
- `TARGET_COL = "target_word_class"`;
- `N_COMPONENTS_FULL = 65`, meaning that full PCA is initially computed using all original semantic dimensions;
- `VAR_THRESHOLD = 0.75`, used to determine how many principal components are needed to retain approximately 75% of the variance;
- `K_RANGE = range(2, 16)`, defining the range of cluster numbers tested;
- `K_SEMANTIC = 3`, used to compare clustering results with the three known target classes.

### C.1.3 Loading the standardized data

The function `load_data()` loads the standardized training set. It separates the identifier and target columns from the semantic predictors. The output is composed of:

- the full `DataFrame`;
- the feature matrix  $X$ ;
- the target vector  $y$ , used only for post-hoc comparison.

### C.1.4 PCA routine

The function `run_pca()` fits PCA on the full standardized matrix. It computes:

- the explained variance ratio of each component;
- the cumulative explained variance;
- the number of components needed to reach 75% cumulative explained variance;
- the cumulative variance explained by the first two components;
- the full PCA scores;
- the two-dimensional PCA representation.

The routine also saves the scree plot and the cumulative variance plot. These figures are used to understand whether the semantic space has a low-dimensional structure or whether variance is distributed across many directions.

### C.1.5 PCA visualisation

The function `plot_pca_biplot()` projects lexical items onto PC1 and PC2 and colours the points according to their known lexical-semantic class. This does not make PCA supervised: the labels are used only for visualization.

The function `plot_pca_loadings()` identifies the features that contribute most strongly to PC1 and PC2. This allows the first principal components to be interpreted semantically.

### C.1.6 K-Means clustering

The function `search_k_kmeans()` runs K-Means over a range of possible cluster numbers. For each  $k$ , it computes:

- within-cluster inertia;
- silhouette score;
- cluster assignments.

It also computes the Hartigan index, which evaluates the decrease in inertia when moving from  $k$  to  $k + 1$ . The script saves the elbow plot, silhouette plot, and Hartigan plot.

### C.1.7 Agglomerative Hierarchical Clustering

The function `run_ahc()` performs hierarchical clustering using complete linkage and Euclidean distance. It saves a dendrogram and computes silhouette scores for several values of  $k$ . The clustering solutions are then plotted on the PC1–PC2 plane.

### C.1.8 Comparison with known labels

The function `compare_with_labels()` compares the cluster assignments with the known lexical-semantic labels using:

- Adjusted Rand Index;
- Normalized Mutual Information;
- contingency tables;
- heatmaps of cluster-label correspondence.

This comparison is essential because it shows whether the unsupervised geometry of the data naturally reproduces the *Thing*, *Action*, and *Property* distinction.

### C.1.9 Pseudocode for Phase 2

---

**Algorithm 5** Phase 2: Unsupervised exploration
 

---

```

1: Load standardized training data from Phase 1.3
2: Separate identifier columns from the semantic feature matrix
3: Fit full PCA on the 65 standardized semantic features
4: Compute explained variance and cumulative explained variance
5: Identify the number of components needed to reach 75% variance
6: Save scree plot and cumulative variance plot
7: Project observations onto PC1 and PC2
8: Save PCA biplot and loading plots
9: Use the retained principal components as reduced clustering space
10: for each  $k$  in the selected range do
11:     Fit K-Means
12:     Compute inertia and silhouette score
13: end for
14: Save K-Means elbow and silhouette plots
15: Perform hierarchical clustering with complete linkage
16: for each  $k$  in the selected range do
17:     Cut dendrogram into  $k$  clusters
18:     Compute silhouette score
19: end for
20: Save AHC dendrogram and cluster plots
21: Compare  $k = 3$  cluster labels with known labels
22: Compute ARI and NMI
23: Save contingency tables and heatmaps
24: Save JSON summary
  
```

---

## C.2 Phase 3.1: Supervised data loading and preliminary checks

### C.2.1 File: `phase3_1_supervised_data_loading_checks.py`

This script starts the supervised part of the pipeline. Its purpose is to load the standardized train and test datasets produced by Phase 1.3, separate predictors from the target variable, and create the supervised modelling objects used by the later phases.

The script expects as input:

`s1ld_phase1_3_train_scaled.csv`

and

`s1ld_phase1_3_test_scaled.csv`

The outputs are saved in:

`outputs/III/phase3_1_supervised_data_loading_checks/`

### C.2.2 Purpose of the script

The script produces the core supervised objects:

- $X_{\text{train}}$ , the training feature matrix;
- $X_{\text{test}}$ , the test feature matrix;
- $y_{\text{train}}$ , the training target vector;
- $y_{\text{test}}$ , the test target vector;
- the list of feature names;
- the list of class names.

It also produces quality-control reports on dimensions, missing values, duplicates, duplicated word forms, class distribution, and non-numeric features.

### C.2.3 Path resolution

The function `resolve_phase1_3_dir()` searches multiple candidate directories for the Phase 1.3 train and test files. This makes the script robust to small differences in folder structure. If the required files are not found, the function raises a `FileNotFoundError`.

### C.2.4 Loading data

The function `load_data()` loads the train and test CSV files. These files already contain identifier columns, target labels, and standardized numerical semantic features.

### C.2.5 Structural validation

The function `validate_required_columns()` checks whether the required columns `entry_id`, `word`, and `target_word_class` exist in both train and test data.

The function `extract_xy()` defines the predictor columns as all columns except the identifier and target columns. It then checks that all predictors are numeric. If any non-numeric predictor is found, the script saves a report and stops, because the subsequent supervised models require numerical matrices.

The function `check_feature_alignment()` verifies that train and test contain the same feature columns in the same order. This is essential because the model fitted on train data must receive the same feature structure at test time.

### C.2.6 Diagnostic reports

The function `build_shape_summary()` creates a summary table with the number of observations, total columns, feature columns, missing cells, exact duplicates, duplicated entry IDs, and duplicated word forms.

The function `build_class_distribution()` computes the target distribution separately for train, test, and total data.

The function `build_missing_values_report()` reports any remaining missing values. Ideally, after Phase 1.3, no missing values should remain.

The function `build_duplicate_reports()` distinguishes exact duplicate rows from duplicated word forms. Exact duplicates are potential data-quality problems, while duplicated word forms may correspond to distinct lexical entries.

### C.2.7 Plots

The script creates:

- a class-distribution bar plot;
- a missing-values plot by split;
- a top-missingness plot;
- a duplicate-diagnostics plot.

### C.2.8 Pseudocode for Phase 3.1

---

**Algorithm 6** Phase 3.1: Supervised data loading and checks
 

---

- 1: Locate Phase 1.3 train and test files
  - 2: Load scaled training and test datasets
  - 3: Check that identifier and target columns are present
  - 4: Define predictors as all non-identifier columns
  - 5: Extract  $X_{\text{train}}, y_{\text{train}}, X_{\text{test}}, y_{\text{test}}$
  - 6: Check that all predictors are numeric
  - 7: Check that train and test feature columns match
  - 8: Compute dataset shape summary
  - 9: Compute class distributions
  - 10: Compute missing-value report
  - 11: Detect exact duplicate rows
  - 12: Detect duplicated word forms
  - 13: Save  $X, y$ , feature names, and class names
  - 14: Save diagnostic reports and plots
  - 15: Save JSON summary
- 

## C.3 Phase 3.2: Multinomial logistic baseline

### C.3.1 File: `phase3_2_train_test_baseline.py`

This script fits the first supervised classifier on the original semantic feature space. The model is a multinomial logistic regression, used as a baseline for all later models.

The input files are the  $X, y$ , feature-name, and class-name outputs generated by Phase 3.1. The outputs are saved in:

`outputs/III/phase3_2_train_test_baseline/`

### C.3.2 Why a multinomial logistic baseline is used

The response variable has three mutually exclusive classes. Therefore, multinomial logistic regression is the natural extension of binary logistic regression to this setting. It provides a transparent statistical baseline and allows the later regularized models to be compared against an interpretable GLM reference.

### C.3.3 Data loading and checks

The function `load_phase3_1_outputs()` loads the train/test matrices and target vectors from Phase 3.1.



The function `check_feature_alignment()` verifies that the column order matches the saved feature-name metadata.

The function `check_shapes_and_classes()` verifies that:

- `X` and `y` have the same number of rows;
- train and test contain all expected classes;
- the split is valid for supervised multiclass learning.

The function `check_feature_standardization()` verifies that the feature matrices are standardized as expected.

### C.3.4 Baseline model fitting

The function `fit_baseline_model()` fits a multinomial logistic regression model. It first attempts an unpenalized model, which is the closest to a pure GLM baseline. If the installed version of `scikit-learn` does not support the syntax, the script tries compatibility alternatives, including a weakly penalized L2 fallback.

### C.3.5 Prediction and evaluation

The function `predict_with_probabilities()` produces:

- predicted labels;
- predicted class probabilities;
- correctness indicators;
- maximum predicted probability.

The function `compute_split_metrics()` computes:

- accuracy;
- balanced accuracy;
- macro precision;
- macro recall;
- macro F1;
- weighted precision;
- weighted recall;
- weighted F1.

The script also saves classification reports and confusion matrices.

### C.3.6 Cross-validation

The function `run_cross_validation()` performs stratified cross-validation on the training set. This is not used to replace the held-out test evaluation. Instead, it provides a stability diagnostic for the baseline model.

### C.3.7 Coefficient extraction

The function `save_coefficients()` saves the multinomial coefficient matrix and identifies the strongest positive and negative coefficients for each class. This provides an initial interpretation of which semantic features push predictions toward each class.

### C.3.8 Pseudocode for Phase 3.2

---

**Algorithm 7** Phase 3.2: Baseline multinomial logistic regression
 

---

- 1: Load  $X_{\text{train}}, y_{\text{train}}, X_{\text{test}}, y_{\text{test}}$
  - 2: Validate feature alignment
  - 3: Validate class availability
  - 4: Check feature standardization
  - 5: Fit multinomial logistic regression on training data
  - 6: Predict labels and probabilities on train and test data
  - 7: Compute global metrics for train and test
  - 8: Compute class-wise metrics
  - 9: Compute confusion matrices
  - 10: Run stratified cross-validation on training data
  - 11: Save model coefficients
  - 12: Save predictions, metrics, plots, model object, and JSON summary
- 

## C.4 Phase 3.3: Multicollinearity diagnostics

### C.4.1 File: `phase3_3_multicollinearity_analysis.py`

This script investigates redundancy and linear dependence among semantic predictors. It is performed before feature screening and regularized modelling, because multicollinearity affects coefficient stability and feature selection.

The outputs are stored in:

`outputs/III/phase3_3_multicollinearity_analysis/`

### C.4.2 Correlation diagnostics

The function `compute_correlation_diagnostics()` computes:

- the Pearson correlation matrix;
- the absolute correlation matrix;
- all ranked feature pairs by absolute correlation;
- highly correlated pairs;
- counts of feature pairs above selected thresholds.

The script uses several thresholds, including 0.50, 0.70, 0.80, 0.90, and 0.95. A high absolute correlation indicates that two features contain overlapping information.

### C.4.3 VIF and partial $R^2$

The function `compute_vif_table()` computes the Variance Inflation Factor for each feature. For each feature  $X_j$ , the script fits an auxiliary regression:

$$X_j \sim X_{-j},$$

where  $X_{-j}$  denotes all other predictors. It then computes:

$$VIF_j = \frac{1}{1 - R_j^2}.$$

A high VIF indicates that a feature can be well explained by the other features, and is therefore involved in multicollinearity.

The script also computes the associated partial  $R^2$ , defined as:

$$R_{\text{partial},j}^2 = 1 - \frac{1}{VIF_j}.$$

#### C.4.4 Feature clustering

The function `compute_feature_clusters()` clusters the variables using correlation distance:

$$d(X_j, X_k) = 1 - |\text{cor}(X_j, X_k)|.$$

This means that strongly positively or negatively correlated variables are considered close. Complete linkage is used to identify groups of mutually correlated features.

#### C.4.5 Redundant-feature candidates

The function `compute_redundant_feature_candidates()` creates a conservative diagnostic list of features that show signs of redundancy. A feature is considered potentially redundant if it belongs to a non-singleton correlation cluster, has high pairwise correlations, or has elevated VIF.

This list is not used to automatically remove variables. It is only a diagnostic input for later interpretation.

#### C.4.6 Pseudocode for Phase 3.3

---

**Algorithm 8** Phase 3.3: Multicollinearity diagnostics

---

- 1: Load supervised feature matrices from Phase 3.1
  - 2: Compute Pearson correlation matrix on  $X_{\text{train}}$
  - 3: Compute absolute correlation matrix
  - 4: Extract all upper-triangular feature pairs
  - 5: Rank pairs by absolute correlation
  - 6: Count feature pairs above correlation thresholds
  - 7: **for** each feature  $X_j$  **do**
  - 8:     Regress  $X_j$  on all other features
  - 9:     Compute  $R_j^2$
  - 10:    Compute  $VIF_j = 1/(1 - R_j^2)$
  - 11:    Compute partial  $R^2$
  - 12: **end for**
  - 13: Define feature distance as  $1 - |\text{correlation}|$
  - 14: Run hierarchical clustering of features
  - 15: Cut dendrogram at selected correlation threshold
  - 16: Identify redundant-feature candidates
  - 17: Save tables, figures, dendrogram, and JSON summary
-

## C.5 Phase 3.4: Feature screening

### C.5.1 File: `phase3_4_feature_screening.py`

This script ranks predictors according to marginal supervised relevance. Since the current original feature space contains only 65 predictors, the screening step mainly provides ranking and diagnostics. However, the code is designed to remain valid if the feature space is later expanded to hundreds or thousands of predictors.

### C.5.2 ANOVA F-score

The function `compute_anova_scores()` computes the ANOVA F-score for each feature. This measures how strongly the feature values differ across the target classes.

### C.5.3 Mutual information

The function `compute_mutual_information_scores()` computes mutual information between each feature and the target. Mutual information is less restricted than ANOVA because it can capture more general dependencies.

### C.5.4 Univariate logistic score

The function `compute_univariate_logistic_scores()` fits one logistic regression model per feature. Each model uses only a single predictor. The feature is then scored according to its univariate predictive performance.

### C.5.5 Combined ranking

The function `combine_rankings()` normalizes the different score types and combines them into one global screening score:

$$0.40 \times \text{ANOVA} + 0.40 \times \text{Mutual Information} + 0.20 \times \text{Univariate Logistic Score}.$$

This combined score is used to rank features. If the number of features is greater than 500, only the top 500 are retained. If the number of features is 65, all features are retained but ordered by importance.

### C.5.6 Pseudocode for Phase 3.4

---

**Algorithm 9** Phase 3.4: Feature screening
 

---

```

1: Load supervised train and test matrices
2: for each feature do
3:   Compute ANOVA F-score
4:   Compute mutual information with target
5:   Fit univariate logistic model
6:   Compute univariate logistic score
7: end for
8: Normalize all score types
9: Combine scores into one screening score
10: Rank features by combined score
11: if number of features is greater than 500 then
12:   Keep top 500 features
13: else
14:   Keep all features in ranked order
15: end if
16: Compute overlap among rankings
17: Save rankings, selected feature names, diagnostic plots, and JSON summary
  
```

---

## C.6 Phase 3.5: Regularized multinomial models

### C.6.1 File: `phase3_5_regularized_models.py`

This script fits the main regularized supervised models: LASSO and Elastic Net multinomial logistic regression. The models are fitted on the screened feature set from Phase 3.4.

### C.6.2 LASSO model

The function `fit_lasso()` uses `LogisticRegressionCV` with an L1 penalty. The regularization parameter is selected by stratified cross-validation using macro F1 as the scoring criterion.

LASSO is important because it can set coefficients exactly to zero, thereby selecting features automatically.

### C.6.3 Elastic Net model

The function `fit_elastic_net()` uses a mixed L1/L2 penalty. The L1 component induces sparsity, while the L2 component stabilizes the model when features are correlated.

This is particularly important in this dataset because Phase 3.3 showed substantial multicollinearity among semantic features.

### C.6.4 Model evaluation

The function `evaluate_model()` computes train and test metrics. The script saves predictions, classification reports, confusion matrices, coefficient tables, selected features, CV scores, fitted model objects, and plots.

### C.6.5 Pseudocode for Phase 3.5

---

**Algorithm 10** Phase 3.5: LASSO and Elastic Net
 

---

```

1: Load  $X_{\text{train}}, y_{\text{train}}, X_{\text{test}}, y_{\text{test}}$ 
2: Load screened feature list from Phase 3.4
3: Subset train and test matrices to screened features
4: Define grid of inverse regularization strengths  $C$ 
5: Fit LASSO multinomial logistic regression with cross-validation
6: Fit Elastic Net multinomial logistic regression with cross-validation
7: for each fitted model do
8:   Predict train and test labels
9:   Compute evaluation metrics
10:  Save test predictions
11:  Save classification report
12:  Save confusion matrix
13:  Save coefficient table
14:  Identify selected features
15:  Save selected feature names
16:  Save CV scores and CV plots
17: end for
18: Save fitted models and JSON summary
  
```

---

## C.7 Phase 3.6: Sparsity analysis

### C.7.1 File: `phase3_6_sparsity_analysis.py`

This script does not fit new models. Instead, it analyses the coefficient tables produced by Phase 3.5 to quantify sparsity.

### C.7.2 Sparsity summaries

The function `summarize_sparsity()` counts:

- total coefficients;
- zero coefficients;
- non-zero coefficients;
- zero share;
- active features;
- inactive features.

A coefficient is considered non-zero if its absolute value is greater than a small tolerance.

### C.7.3 Class-level sparsity

The function `summarize_by_class()` computes the number of non-zero coefficients separately for each class. This helps identify whether the model uses different semantic dimensions to predict different lexical classes.

### C.7.4 Shared and model-specific selected features

The function `shared_specific()` compares features selected by LASSO and Elastic Net. Features can be:

- selected by both models;
- selected only by LASSO;
- selected only by Elastic Net;
- inactive in both.

### C.7.5 Pseudocode for Phase 3.6

---

**Algorithm 11** Phase 3.6: Sparsity analysis
 

---

```

1: Load LASSO coefficient table
2: Load Elastic Net coefficient table
3: for each coefficient do
4:   Mark as active if absolute value is above tolerance
5: end for
6: for each model do
7:   Count zero and non-zero coefficients
8:   Count active and inactive features
9:   Compute sparsity proportions
10: end for
11: for each class do
12:   Count non-zero coefficients
13:   Compute class-specific sparsity
14: end for
15: Compare LASSO and Elastic Net selected features
16: Identify shared and model-specific features
17: Build coefficient matrices
18: Save tables, heatmaps, sparsity plots, and JSON summary
  
```

---

## C.8 Phase 3.7: Final model evaluation

### C.8.1 File: `phase3_7_final_model_evaluation.py`

This script integrates the previous supervised results and constructs the final post-selection classifier.

### C.8.2 Post-selection logic

The script loads the selected features from LASSO and Elastic Net and takes their union. A new multinomial logistic regression model is then fitted on this reduced feature set.

This step separates feature selection from final coefficient estimation. The regularized models identify the relevant variables, while the post-selection model refits the classifier using only those selected variables.

### C.8.3 Model comparison

The script compares:

- baseline multinomial logistic regression;
- LASSO;
- Elastic Net;
- post-selection logistic regression.

The function `choose_final_model()` selects the best model according to:

1. test macro F1;
2. test balanced accuracy;
3. test accuracy;
4. smaller number of features.

### C.8.4 Pseudocode for Phase 3.7

---

**Algorithm 12** Phase 3.7: Final model evaluation
 

---

- 1: Load supervised train and test matrices
  - 2: Load fitted LASSO and Elastic Net models
  - 3: Load selected feature lists from both regularized models
  - 4: Define final selected feature set as their union
  - 5: Fit post-selection multinomial logistic regression
  - 6: Evaluate LASSO, Elastic Net, and post-selection model
  - 7: Add baseline metrics from Phase 3.2 if available
  - 8: Compare models on test macro F1, balanced accuracy, accuracy, and feature count
  - 9: Select final model
  - 10: Save final predictions, confusion matrices, coefficients, selected features, plots, model object, and JSON summary
- 

## C.9 Phase 3-extra: Alternative classifiers

### C.9.1 File: `phase3_extra_classifiers_comparison.py`

This script tests whether non-linear or non-parametric classifiers improve over the final linear classifier. It uses the same selected feature space as Phase 3.7, so that the comparison focuses on the classifier rather than on preprocessing or feature selection.

### C.9.2 Classifiers tested

The script evaluates:

- Dummy majority classifier;
- post-selection logistic regression;
- linear SVM;
- RBF-kernel SVM;
- Random Forest;
- k-nearest neighbors.

### C.9.3 Model grid

The function `build_model_grid()` defines each classifier and its hyperparameter grid. The grids are intentionally compact because the dataset is small and the goal is controlled comparison rather than exhaustive tuning.

### C.9.4 Evaluation

For each model, the script performs cross-validated hyperparameter tuning when a parameter grid is defined. It then evaluates the best estimator on train and test data.



### C.9.5 Pseudocode for Phase 3-extra

---

**Algorithm 13** Phase 3-extra: Alternative classifiers
 

---

```

1: Load scaled train and test datasets
2: Load selected feature list from Phase 3.7
3: if selected feature file is unavailable then
4:   Use all numeric semantic features
5: end if
6: Extract  $X_{\text{train}}$ ,  $X_{\text{test}}$ ,  $y_{\text{train}}$ ,  $y_{\text{test}}$ 
7: Define classifiers and hyperparameter grids
8: for each classifier do
9:   if hyperparameter grid is not empty then
10:    Run GridSearchCV using macro F1
11:    Select best estimator
12:   else
13:    Fit classifier directly
14:   end if
15:   Predict train and test labels
16:   Compute accuracy, balanced accuracy, macro F1, and weighted F1
17:   Save classification report
18:   Save confusion matrix
19:   Save predictions and fitted model
20: end for
21: Save model comparison tables and plots
22: Select best model by test macro F1
23: Save JSON summary

```

---

## C.10 Phase 4: Supervised classification using clustering labels

### C.10.1 File: `phase4_cluster_label_supervised_analysis.py`

This script repeats the supervised analysis using clustering-derived labels instead of the original lexical-semantic labels. The goal is to test whether data-driven clusters are easier to predict than the externally defined classes.

### C.10.2 Cluster-target construction

The script reconstructs the clustering target using the same logic as Phase 2: PCA is applied to the standardized semantic feature space, and K-Means is fitted on the PCA-reduced training data. The fitted K-Means model is then used to assign labels to both train and test observations.

For the main comparison, the script uses  $k = 3$ , so that the cluster-based target has the same number of classes as the original *Thing*, *Action*, *Property* target.

### C.10.3 Repeated supervised pipeline

For each cluster target, the script performs:

- target distribution analysis;
- comparison between cluster labels and original labels;
- feature screening;

- baseline logistic regression;
- LASSO;
- Elastic Net;
- post-selection logistic regression;
- final comparison.

#### C.10.4 Pseudocode for Phase 4

---

**Algorithm 14** Phase 4: Supervised classification with clustering labels
 

---

- 1: Load Phase 3 supervised matrices and original labels
  - 2: Load Phase 2 PCA and clustering decisions when available
  - 3: Fit PCA on training features
  - 4: Transform training and test features
  - 5: Fit K-Means on training PCA scores
  - 6: Assign cluster labels to train and test observations
  - 7: Save cluster assignments and target distributions
  - 8: Compare cluster labels with original labels
  - 9: **for** each cluster target **do**
  - 10:     Run feature screening
  - 11:     Fit baseline logistic classifier
  - 12:     Fit LASSO model
  - 13:     Fit Elastic Net model
  - 14:     Fit post-selection logistic model
  - 15:     Evaluate all models
  - 16:     Save metrics, coefficients, selected features, confusion matrices, and plots
  - 17: **end for**
  - 18: Compare cluster-label performance with original-label performance
  - 19: Save JSON summary
- 

### C.11 Overall logic of the post-preprocessing pipeline

The complete pipeline after preprocessing follows a progressive statistical logic.

First, Phase 2 explores the geometry of the feature space without using the target labels for model fitting. This reveals whether the semantic feature space contains natural clusters or low-dimensional latent structure.

Second, Phase 3.1 and Phase 3.2 establish a supervised baseline. The baseline is necessary because all later models must be judged relative to a simple, interpretable classifier.

Third, Phase 3.3 diagnoses multicollinearity. This justifies the later use of LASSO and Elastic Net, because correlated semantic features can destabilize ordinary coefficient estimates.

Fourth, Phase 3.4 ranks features by marginal supervised relevance. This prepares the feature space for regularized modelling and becomes especially important when the predictor space is expanded.

Fifth, Phase 3.5 and Phase 3.6 fit and interpret sparse regularized models. These phases connect prediction with interpretability by identifying which semantic dimensions are actually used by the classifier.

Sixth, Phase 3.7 selects the final linear model using a transparent rule based primarily on macro F1 and balanced accuracy.

Seventh, Phase 3-extra tests whether more flexible classifiers improve performance, thereby evaluating

whether the relationship between semantic features and lexical labels is purely linear or involves non-linear interactions.

Finally, Phase 4 repeats the supervised analysis using clustering-derived labels. This provides a direct comparison between externally defined lexical-semantic categories and data-driven semantic groupings.

---

**Algorithm 15** Complete analysis workflow after preprocessing

---

- 1: Load standardized train and test datasets
  - 2: Explore semantic structure with PCA and clustering
  - 3: Compare unsupervised clusters with original labels
  - 4: Prepare supervised matrices  $X$  and target vectors  $y$
  - 5: Fit multinomial logistic baseline
  - 6: Diagnose multicollinearity using correlations, VIF, and dendrograms
  - 7: Rank predictors with supervised screening
  - 8: Fit LASSO and Elastic Net multinomial models
  - 9: Analyse sparsity and selected features
  - 10: Fit final post-selection logistic model
  - 11: Compare all linear supervised models
  - 12: Fit alternative classifiers on selected features
  - 13: Compare linear and non-linear predictive performance
  - 14: Rebuild cluster-based targets
  - 15: Repeat supervised classification with clustering labels
  - 16: Compare original-label and cluster-label predictability
- 

## C.12 Additional tables and figures

